ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN IoT

HỆ THỐNG PHÂN LOẠI KIỆN HÀNG THÔNG MINH

KẾT HỢP MÃ QR VÀ AI KIỂM TRA BAO BÌ, CÓ BẢO MẬT

(Đề tài A — Smart Logistics Sorter)

Thiết bị chính: Raspberry Pi 4 + Pi Camera (Picamera2, CSI) · ESP32 (ESP-IDF v5.x) · ThingsBoard CE

Bảo mật: X.509 mutual TLS · HMAC-SHA256 · Secure OTA ECDSA P-256 · Secure Boot V2 · Isolation Forest

Sinh viên thực hiện: Chính Lê và nhóm

Giảng viên hướng dẫn: ……

Đà Nẵng, tháng 10 năm 2026

MỤC LỤC

MỤC LỤC 2
1. Giới thiệu 4
1.1 Bối cảnh và lý do chọn đề tài 4
1.2 Mục tiêu 4
1.3 Phạm vi và giả định 4
1.4 Cấu trúc báo cáo 5
2. Phân tích bài toán 5
2.1 Vai trò của QR và AI 5
2.2 Luật phân loại 5
2.3 Yêu cầu phi chức năng 6
3. Kiến trúc hệ thống và phần cứng 7
3.1 Tổng quan bốn tầng 7
3.2 Các khối chính 8
3.3 Danh sách linh kiện 8
3.4 Sơ đồ chân ESP32 9
3.5 Bố trí cơ khí dọc băng chuyền 10
3.6 Luồng xử lý một kiện (dừng rồi chụp) 10
4. Giao thức và định dạng dữ liệu 11
4.1 Định dạng mã QR 12
4.2 Khung UART Pi ↔ ESP32 12
4.3 MQTT với ThingsBoard 14
4.4 Whitelist RPC 15
4.5 Manifest Secure OTA 15
4.6 File cấu hình của Pi 16
5. Hiện thực chi tiết từng mô-đun 16
5.1 P1 — Khung repo và giao thức dùng chung 16
5.2 P2 — Firmware ESP32 phần chấp hành 17
5.3 P3 — sorter-core trên Raspberry Pi 18
5.4 P4 — AI kiểm tra bao bì 20
5.5 P5 — ThingsBoard, MQTT TLS, RPC và dashboard 22
5.6 P6 — Hardening Raspberry Pi và bảo vệ ESP32 22
5.7 P7 — Ứng dụng Secure OTA trên máy tính 23
5.8 P8 — Secure OTA phía ESP32 23
5.9 P9 — Phát hiện truy cập bất thường 25
5.10 P10 — Tích hợp, kiểm thử demo, số liệu 25

6. Bảo mật tổng hợp 26
Giới hạn được nêu trung thực 27
7. Kiểm thử và kết quả 28
7.1 Bộ kiểm thử tự động 28
7.2 Chạy mô phỏng toàn pipeline 28
7.3 Kịch bản demo ở chế độ mô phỏng 29
7.4 Benchmark và số liệu 30
7.5 Đối chiếu tiêu chí nghiệm thu 30
8. Hướng dẫn cài đặt, vận hành và demo 30
8.1 Chạy nhanh trên laptop (không cần phần cứng) 31
8.2 Cài đặt Raspberry Pi 31
8.3 Nạp ESP32 31
8.4 ThingsBoard 31
8.5 Kịch bản demo bắt buộc 32
8.6 Chuẩn bị trước buổi demo 32
9. Kết luận 32
9.1 Những gì đã hoàn thành 32
9.2 Việc còn lại trước ngày chấm 33
9.3 Hướng phát triển 33
Phụ lục A — Cấu trúc mã nguồn 33
Phụ lục B — Truy vết chức năng 34
Phụ lục C — Tài liệu tham khảo 35

1. Giới thiệu

1.1 Bối cảnh và lý do chọn đề tài

Trong kho vận hiện đại, kiện hàng đi qua băng chuyền cần được phân loại tự động theo điểm đến và đồng thời được kiểm tra tình trạng bao bì trước khi xuất kho. Hai việc này có bản chất khác nhau: điểm đến là thông tin đã biết trước và có thể mã hóa sẵn trên nhãn, còn tình trạng vật lý của hộp (rách, móp, vỡ, ướt bẩn) chỉ biết được tại thời điểm hộp đi qua trạm kiểm tra. Đề tài xây dựng một mô hình phần cứng chạy thật kết hợp hai cơ chế đó: **mã QR có chữ ký số** trả lời câu hỏi "hàng đi đâu" và **AI thị giác máy tính** trả lời câu hỏi "hàng có được đi không".

Bên cạnh chức năng phân loại, hệ thống IoT trong kho còn là mục tiêu tấn công: QR giả để chuyển hướng hàng, lệnh điều khiển giả qua mạng, nạp firmware độc hại lên bộ điều khiển. Vì vậy đề tài đặt bảo mật làm trụ cột thứ ba, với xác thực thiết bị bằng chứng chỉ X.509, chữ ký HMAC cho mã QR, whitelist lệnh điều khiển, Secure OTA có chữ ký ECDSA và chống rollback, cùng một mô-đun học máy phát hiện truy cập bất thường.

1.2 Mục tiêu

- Xây dựng mô hình phần cứng hoàn chỉnh: băng chuyền, cảm biến, camera, cơ cấu gạt 3 hướng (khay A, khay B, khay Loại), đèn, còi, nút dừng khẩn cấp.
- Đọc mã QR có chữ ký HMAC-SHA256 và chống dùng lại (replay); tự nhận diện lỗi bao bì bằng mô hình phân loại ảnh chạy trên Raspberry Pi 4.
- Gửi và nhận dữ liệu với ThingsBoard qua MQTT TLS 8883 bằng chứng chỉ X.509; dashboard có bộ đếm, log, ảnh hàng lỗi và nút dừng khẩn cấp.
- Cơ chế xác thực và bảo mật nhiều lớp, có thể chứng minh trực tiếp khi demo.
- Điểm cộng: ứng dụng nạp firmware bảo mật (Secure OTA) và phát hiện truy cập bất thường bằng Isolation Forest.

1.3 Phạm vi và giả định

- Mỗi thời điểm chỉ có một kiện trong vùng phân loại; kiện sau cách kiện trước ít nhất bằng quãng từ camera đến cuối băng.
- Hộp carton nhỏ, QR dán mặt trên hướng về camera; đèn chiếu sáng cố định quanh vị trí chụp.
- Pi, ESP32 (khi OTA) và laptop chạy ThingsBoard dùng chung một mạng LAN/Wi-Fi; ThingsBoard CE tự cài bằng Docker để bật MQTT TLS.
- Camera là **Pi Camera Module gắn cổng CSI**, điều khiển bằng thư viện Picamera2; không dùng camera USB.
- Cảm biến hỗ trợ cả TCRT5000 và HC-SR04, chọn bằng cấu hình; servo SG90 hoặc MG996R.

1.4 Cấu trúc báo cáo

Chương 2 phân tích bài toán và luật phân loại. Chương 3 trình bày kiến trúc hệ thống, phần cứng và luồng xử lý. Chương 4 là hợp đồng giao thức và định dạng dữ liệu dùng chung. Chương 5 mô tả hiện thực chi tiết từng mô-đun (P1–P10). Chương 6 tổng hợp các biện pháp bảo mật. Chương 7 trình bày kiểm thử và kết quả. Chương 8 là hướng dẫn cài đặt, vận hành và kịch bản demo. Chương 9 kết luận, nêu việc còn lại và hướng phát triển. Phụ lục liệt kê cấu trúc mã nguồn và bảng truy vết chức năng. Báo cáo có 13 hình (kiến trúc, triển khai, đấu nối, tuần tự, lưu đồ quyết định, QR, khung UART, máy trạng thái, luồng dữ liệu, ảnh mẫu, pipeline AI, Secure OTA, phát hiện bất thường, lớp bảo mật) do nhóm vẽ từ mã nguồn thật trong docs/report/make_diagrams.py.

2. Phân tích bài toán

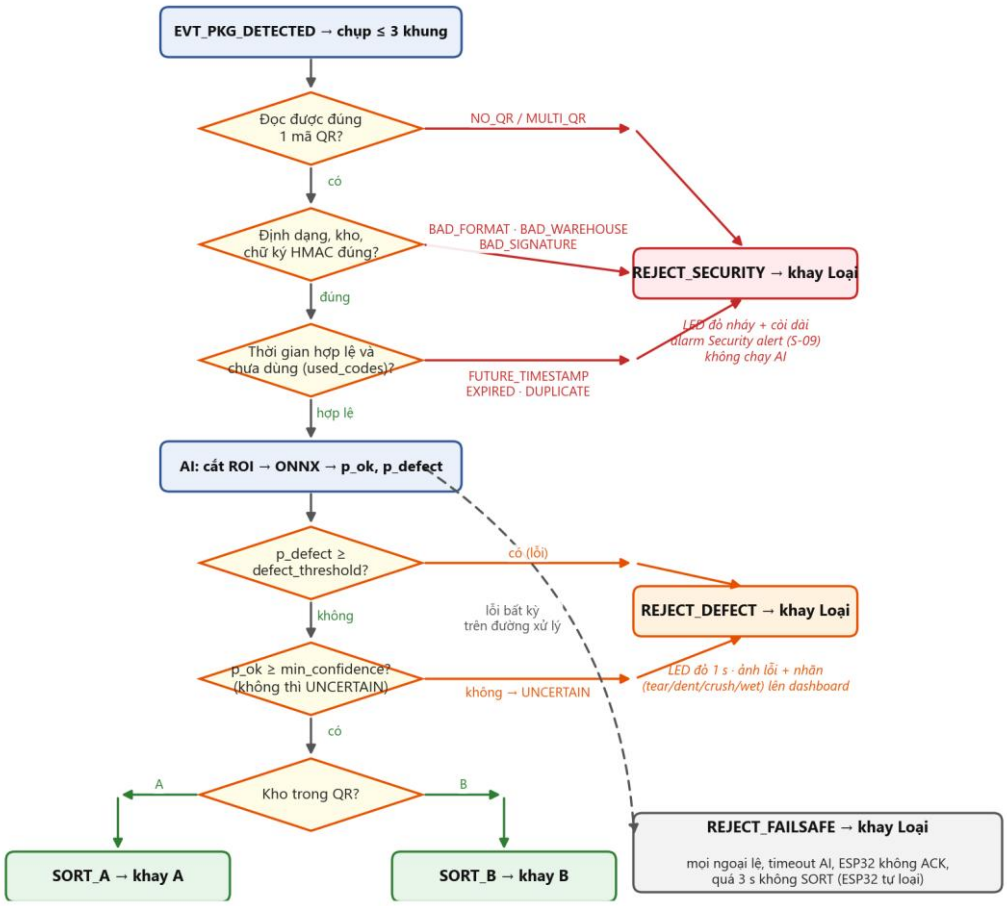
2.1 Vai trò của QR và AI

Mã QR chứa bốn trường: mã kiện, kho đích (A hoặc B), thời gian tạo mã và chữ ký HMAC-SHA256 tính trên ba trường đầu bằng khóa bí mật 32 byte. Chữ ký bảo đảm mã do hệ thống của nhóm phát hành; thời gian và bảng mã đã dùng chống việc in lại hoặc quét lại một mã hợp lệ. QR không thể biết hộp có bị rách trên đường đi hay không, nên AI camera là thành phần không thể thay thế: mô hình phân loại ảnh nhìn vào vùng hộp dưới camera và trả về xác suất bao bì lỗi cùng loại lỗi.

2.2 Luật phân loại

#	QR	AI bao bì	Mã quyết định	Khay	Tín hiệu và dashboard
1	Hợp lệ, kho A	OK	SORT_A	A	LED xanh, còi bíp ngắn, log
2	Hợp lệ, kho B	OK	SORT_B	B	LED xanh, còi bíp ngắn, log
3	Hợp lệ	Lỗi hoặc độ tin cậy dưới ngưỡng	REJECT_DEFECT	Loại	LED đỏ, ảnh lỗi + nhãn lỗi trên dashboard
4	Không đọc được, sai định dạng, chữ ký sai, trùng, hết hạn	không chạy AI	REJECT_SECURITY	Loại	LED đỏ nháy, còi dài, alarm bảo mật
5	Hết thời gian chờ lệnh, mất kết nối, lỗi hệ thống	—	REJECT_FAILSAFE	Loại	LED đỏ, log lỗi

Bảng 2.1 — Luật phân loại (đặc tả mục 1.2)



Hình 2.1 — Lưu đồ quyết định cho một kiện: QR xét trước, AI chỉ chạy khi QR hợp lệ, mọi nhánh không chắc chắn về khay Loại (pi/sorter/decision.py)

Thứ tự xét: kiểm tra QR trước, QR không hợp lệ thì loại ngay và bỏ qua AI. Nguyên tắc fail-safe xuyên suốt: mọi trường hợp không chắc chắn đều vào khay Loại, không bao giờ vào A hoặc B. Về cơ khí, hàng không bị gạt sẽ tự rơi vào khay Loại ở cuối băng, nên servo hỏng hay không có lệnh đều dẫn tới kết quả an toàn.

2.3 Yêu cầu phi chức năng

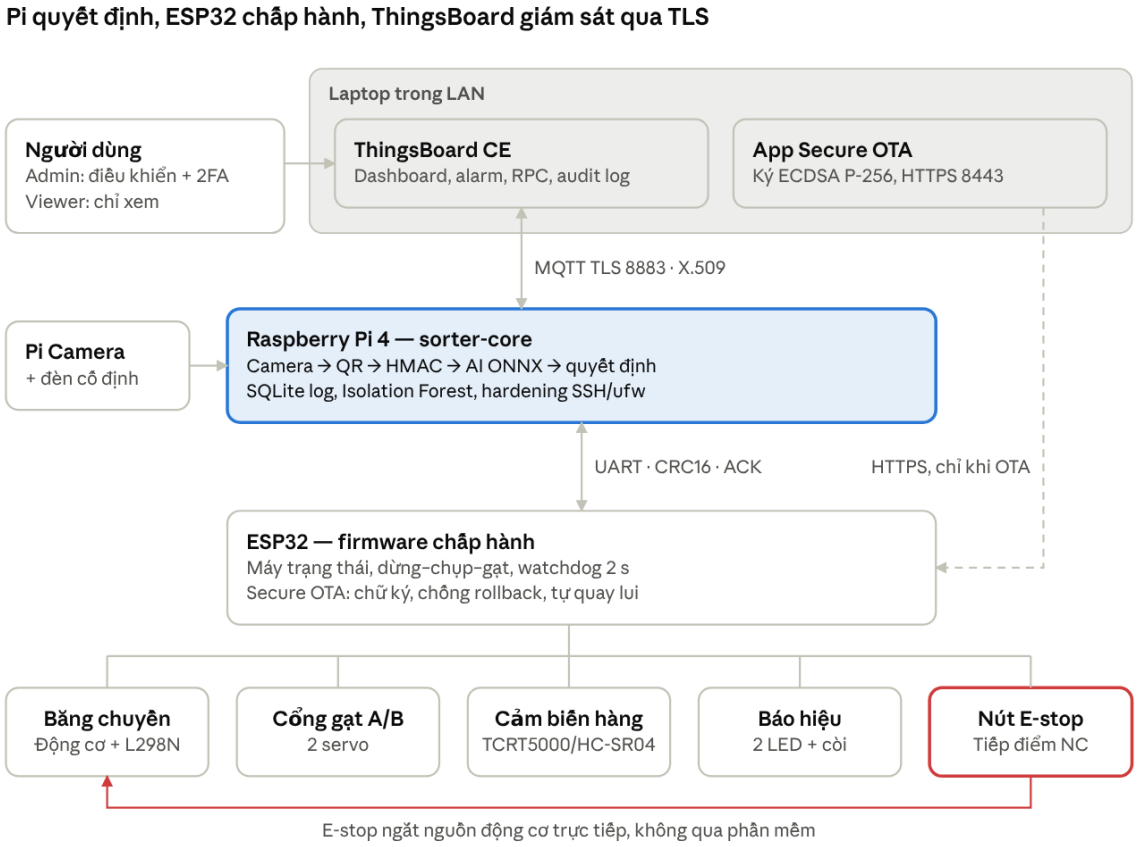
Tiêu chí	Mục tiêu	Cách đo
Độ chính xác AI trên tập test	Accuracy ≥ 90 %, recall lớp Lỗi ≥ 95 %	ai/evaluate.py
Thời gian suy luận trên Pi	≤ 150 ms trung bình	pi/tools/benchmark_model.py (100 lần)
Từ lúc dừng băng đến lệnh SORT	≤ 1 s	latency_ms trung bình 20 kiện
Phân loại đúng khay	20/20 kiện ở kịch bản 1–4	demo_check.py
E-stop từ dashboard	Băng dừng ≤ 0,5 s	Quay video đếm khung hình
QR giả / QR trùng bị chặn	100 %	10 mã giả + 5 lần quét lại
OTA tấn công bị từ chối	100 % cả 3 kiểu	Tab Demo của app OTA
Cảnh báo nhầm Isolation Forest	≤ 1 lần / 30 phút	Log anomaly_flag

Bảng 2.2 — Tiêu chí nghiệm thu (đặc tả mục 5.3)

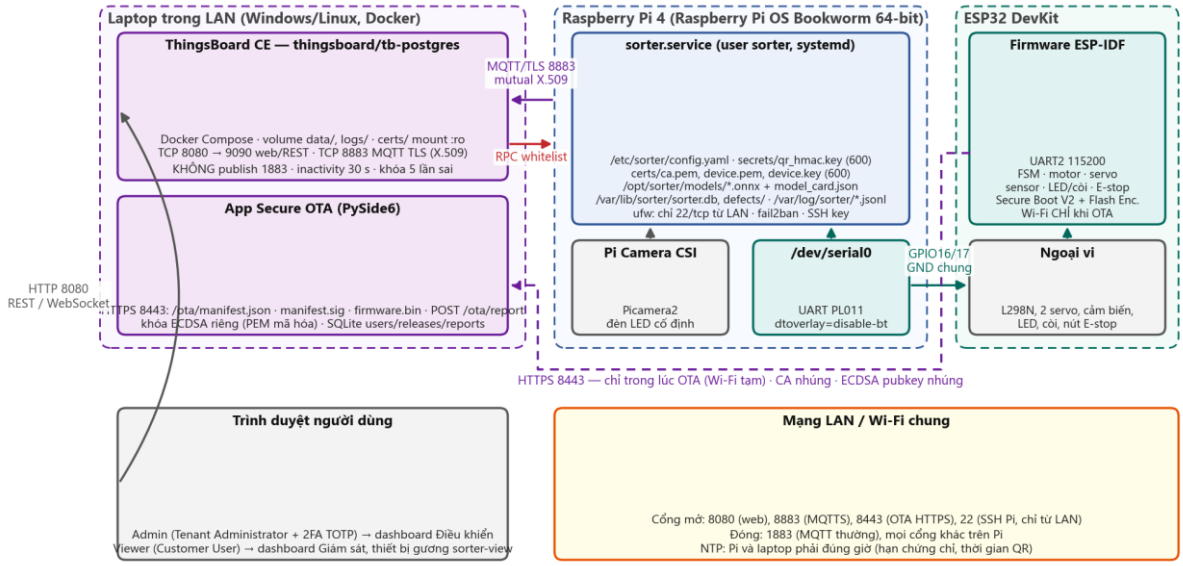
3. Kiến trúc hệ thống và phần cứng

3.1 Tổng quan bốn tầng

Raspberry Pi 4 là bộ não: chụp ảnh, đọc QR, xác thực HMAC, chạy AI, ra quyết định, lưu log SQLite và nói chuyện với ThingsBoard qua MQTT TLS. ESP32 lo phần thời gian thực: băng chuyền, cảm biến, hai servo gạt, đèn, còi, nút dừng khẩn cấp, watchdog liên kết và Secure OTA. Hai bên nối bằng UART 115200 với khung có CRC16, ACK và số thứ tự. ESP32 tự dừng băng khi thấy hàng và tự loại hàng nếu Pi không trả lời trong 3 s, nên hệ thống vẫn an toàn khi Pi treo. Laptop trong LAN chạy ThingsBoard CE (Docker) và ứng dụng Secure OTA; ESP32 chỉ bật Wi-Fi trong lúc OTA.



Hình 3.1 — Kiến trúc hệ thống bốn tầng (Pi quyết định, ESP32 chấp hành, ThingsBoard giám sát qua TLS)



Hình 3.2 — Sơ đồ triển khai: thành phần chạy trên laptop, Raspberry Pi 4 và ESP32; cổng mạng và giao thức giữa chúng

3.2 Các khối chính

Khối	Thiết bị / phần mềm	Nhiệm vụ
Bộ xử lý chính	Raspberry Pi 4 + Pi Camera CSI (Picamera2), Python 3.11	Chụp ảnh, đọc QR, xác thực HMAC, chạy AI ONNX, quyết định, gửi lệnh ESP32, MQTT TLS, SQLite
Bộ chấp hành	ESP32 DevKit, ESP-IDF v5.x, C	Động cơ qua L298N, 2 servo gạt, cảm biến, LED, còi, E-stop, watchdog, Secure OTA
Nền tảng IoT	ThingsBoard CE (Docker trên laptop)	Telemetry, RPC, dashboard, alarm, phân quyền, 2FA
App Secure OTA	Python + PySide6 trên máy dev	Đăng nhập, ký firmware ECDSA P-256, manifest, HTTPS server, lịch sử nạp
Huấn luyện AI	Laptop hoặc Google Colab	Notebook thu dữ liệu 5 nguồn, train ConvNeXt-V2 → chưng cất MobileNetV4, xuất ONNX
Phát hiện bất thường	Python + scikit-learn trên Pi	Isolation Forest trên log RPC, QR, UART, SSH, MQTT

Bảng 3.1 — Các khối chính

3.3 Danh sách linh kiện

Linh kiện	SL	Ghi chú
Raspberry Pi 4 (≥ 2 GB) + thẻ nhớ 32 GB	1	Raspberry Pi OS 64-bit Bookworm
Pi Camera Module v2 hoặc Module 3 (CSI)	1	Gắn cố định nhìn thẳng xuống, cách mặt hộp 20–30 cm; điều khiển bằng Picamera2
ESP32 DevKit (chip revision ≥ v3.0)	2	1 board chính, 1 board dự phòng để thử Secure Boot / Flash Encryption
Động cơ DC giảm tốc + băng chuyền	1	Băng tự chế hoặc bộ có sẵn

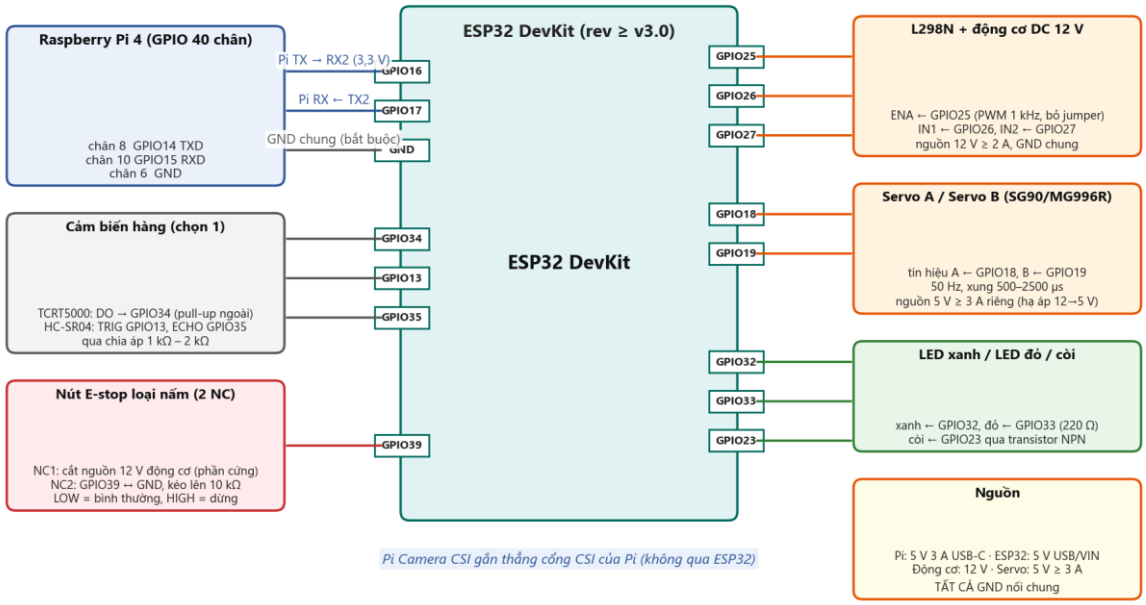
Linh kiện	SL	Ghi chú
Driver L298N	1	Bỏ jumper ENA để điều tốc bằng PWM
Servo SG90 hoặc MG996R	2	Servo A và B, mỗi servo một tay gạt chéo 45°
Cảm biến TCRT5000 hoặc HC-SR04	1	Đặt ngay dưới vùng camera
LED xanh, LED đỏ, còi buzzer	1 mỗi loại	Còi qua transistor NPN nếu dòng > 20 mA
Nút dừng khẩn cấp loại nấm, 2 tiếp điểm NC	1	1 tiếp điểm cắt nguồn động cơ, 1 tiếp điểm báo ESP32
Đèn LED chiếu sáng cố định	1	Rất quan trọng cho độ ổn định của AI
Nguồn 12 V ≥ 2 A; 5 V 3 A USB-C cho Pi; hạ áp 12→5 V ≥ 3 A cho servo	1 mỗi loại	GND nối chung
3 khay hứng (A, B, Loại), hộp carton nhỏ	—	Dùng để chụp dataset và demo

Bảng 3.2 — Linh kiện

3.4 Sơ đồ chân ESP32

Chức năng	Chân ESP32	Nối tới	Ghi chú
UART2 RX / TX	GPIO16 / GPIO17	Pi TX GPIO14 (chân 8) / Pi RX GPIO15 (chân 10)	3,3 V, 115200 8N1, GND chung
PWM động cơ	GPIO25	L298N ENA	LEDC 1 kHz, 10 bit
Chiều động cơ	GPIO26, GPIO27	L298N IN1, IN2	Chỉ chạy một chiều
Servo A / Servo B	GPIO18 / GPIO19	Tín hiệu servo	LEDC 50 Hz, nguồn 5 V riêng
TCRT5000	GPIO34	Chân DO	Chân chỉ đọc, cần pull-up ngoài
HC-SR04	GPIO13 / GPIO35	TRIG / ECHO	ECHO qua cầu chia áp 1 kΩ – 2 kΩ
LED xanh / LED đỏ	GPIO32 / GPIO33	Qua điện trở 220 Ω	
Còi	GPIO23	Qua transistor NPN	
Nút E-stop	GPIO39	Tiếp điểm NC xuống GND, kéo lên 10 kΩ	LOW bình thường; HIGH (bấm hoặc đứt dây) → dừng

Bảng 3.3 — Sơ đồ chân (đặc tả mục 2.3); định nghĩa trong esp32/components/board/include/board.h



Hình 3.3 — Sơ đồ đấu nối ESP32 với Raspberry Pi, cảm biến, nút E-stop, L298N, servo, LED/còi và nguồn

Phía Pi: bật UART phần cứng bằng raspi-config, tắt serial console, thêm `dtoverlay=disable-bt` để dùng UART PL011 ổn định trên `/dev/serial0`.

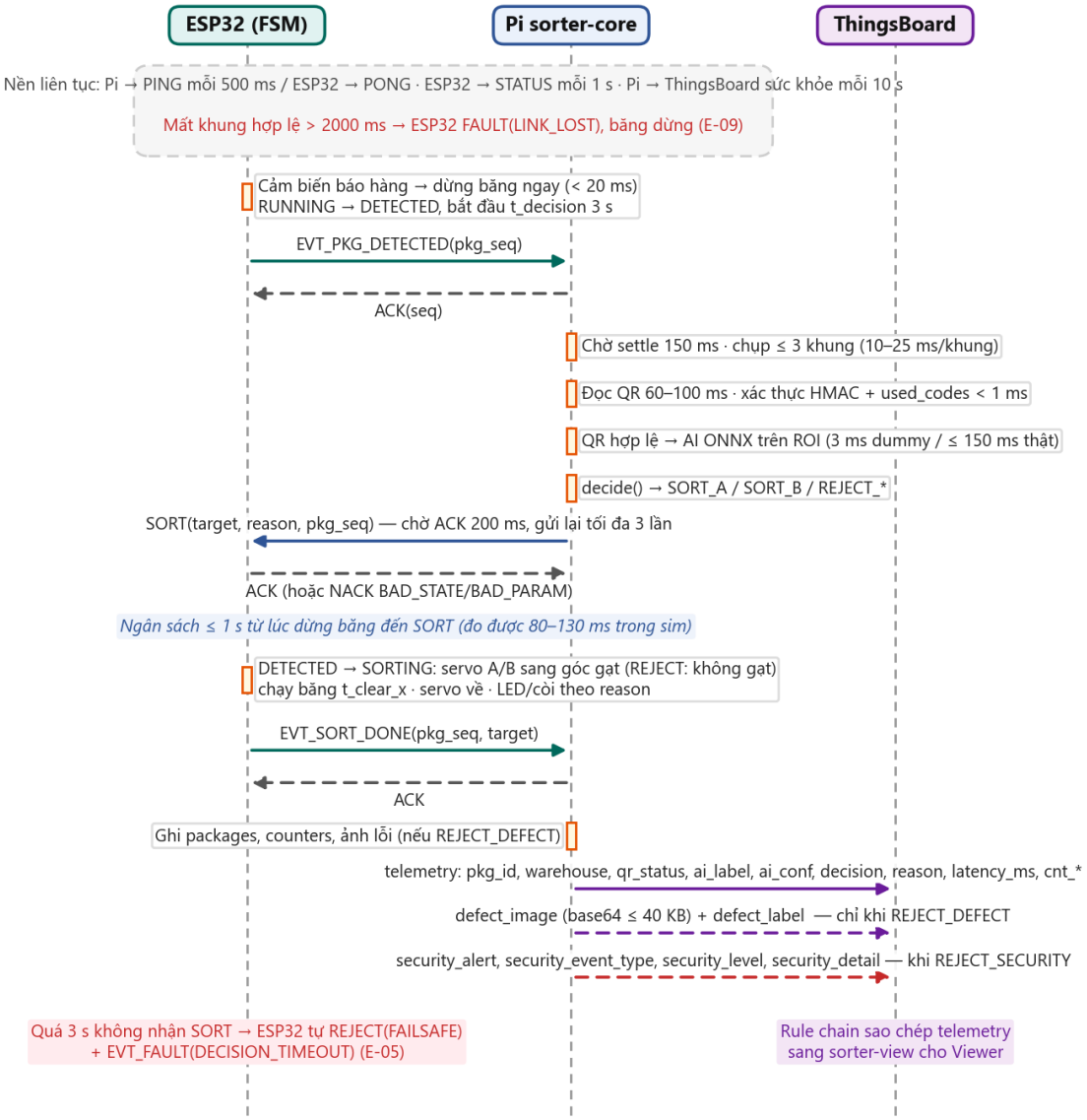
3.5 Bố trí cơ khí dọc băng chuyền

1. Vùng đặt hàng ở đầu băng.
2. Trạm chụp: cảm biến phát hiện hàng nằm ngay dưới camera; đèn LED bao quanh, che ánh sáng ngoài.
3. Cổng gạt A: servo A, tay gạt ở vị trí nghỉ song song mép băng; khi gạt xoay chéo 45° chắn ngang băng, hàng trượt vào khay A bên trái.
4. Cổng gạt B: tương tự, đẩy sang khay B bên phải.
5. Cuối băng: khay Loại. Hàng không bị gạt rơi vào đây (fail-safe cơ khí).

3.6 Luồng xử lý một kiện (dừng rồi chụp)

6. ESP32 đang chạy băng; cảm biến báo có hàng thì ESP32 dừng băng ngay (mục tiêu < 20 ms) và gửi EVT_PKG_DETECTED kèm số thứ tự kiện.
7. Pi chờ băng ổn định 150 ms rồi chụp (tối đa 3 khung nếu chưa đọc được QR).
8. Pi giải mã QR và xác thực: định dạng, kho, chữ ký HMAC, thời gian, chống trùng. Sai thì quyết định ngay REJECT_SECURITY, bỏ qua AI.
9. QR hợp lệ thì Pi cắt ROI và chạy AI phân loại OK/Lỗi.
10. Pi ra quyết định theo bảng 2.1, gửi SORT qua UART, chờ ACK tối đa 200 ms, gửi lại tối đa 3 lần.

11. ESP32 đặt servo, chạy băng trong t_clear, trả servo về, bật LED/còi theo lý do, gửi EVT_SORT_DONE. Quá 3 s không nhận SORT thì tự xử lý như REJECT_FAILSAFE.
12. Pi ghi SQLite, gửi telemetry, ảnh lỗi (nếu có) và cảnh báo lên ThingsBoard.



Hình 3.4 — Sơ đồ tuần tự xử lý một kiện giữa ESP32, Pi và ThingsBoard, kèm mốc thời gian đo được trong mô phỏng

Ngân sách thời gian từ lúc dừng băng đến lúc gửi SORT là ≤ 1 s. Ở chế độ mô phỏng trên laptop, pipeline đo được 80–130 ms cho kiện có QR và khoảng 620–670 ms cho kiện không có QR (phải thử đủ 3 khung).

4. Giao thức và định dạng dữ liệu

Chương này là "hợp đồng" chung giữa Pi, ESP32, ThingsBoard và app OTA. Toàn bộ định nghĩa giao thức UART nằm trong một file duy nhất `shared/protocol/protocol.yaml`;

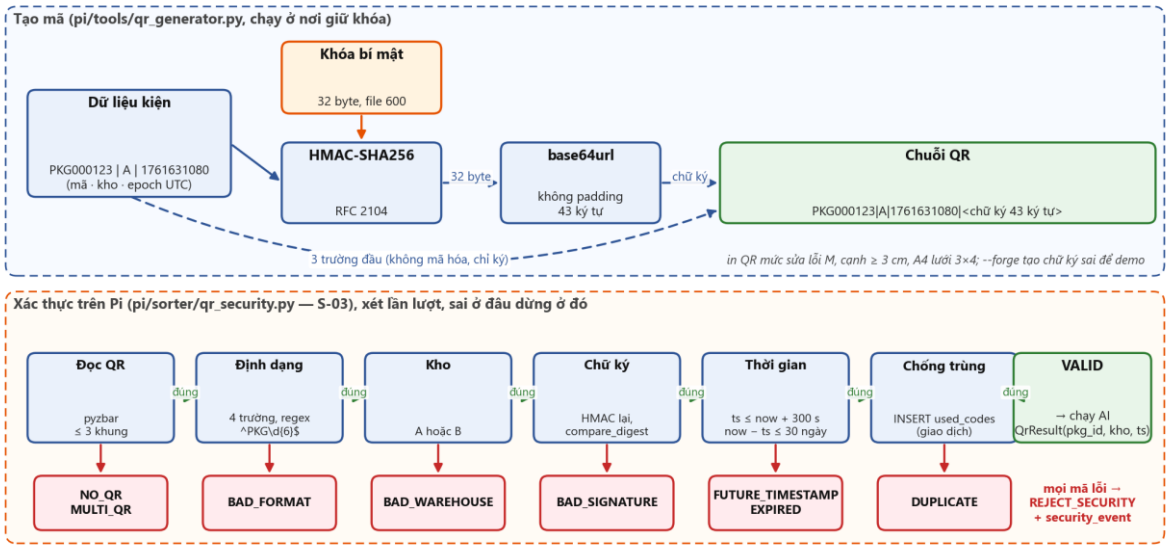
script `gen.py` sinh ra `protocol_gen.h` cho C và `protocol_gen.py` cho Python nên hai bên không thể lệch nhau.

4.1 Định dạng mã QR

```
PKG000123|A|1761631080|<chữ ký 43 ký tự base64url>
```

Trường	Quy tắc
mã_kiện	PKG + 6 chữ số, regex ^PKG\d{6}$
kho	A hoặc B
thời_gian	Unix epoch giây (UTC) lúc tạo mã
chữ_ký	base64url không padding của HMAC-SHA256(khóa 32 byte, "PKG000123|A|1761631080") — 43 ký tự

Bảng 4.1 — Định dạng QR. Mã kết quả xác thực: VALID, NO_QR, MULTI_QR, BAD_FORMAT, BAD_WAREHOUSE, BAD_SIGNATURE, FUTURE_TIMESTAMP, EXPIRED, DUPLICATE



Hình 4.1 — Tạo mã QR có chữ ký HMAC-SHA256 và chuỗi kiểm tra trên Pi với 9 mã kết quả

4.2 Khung UART Pi ↔ ESP32

Trường	Kích thước	Giá trị
SOF	2 byte	0xAA 0x55
VER	1 byte	0x01
SEQ	1 byte	0–255, tăng vòng, mỗi bên một bộ đếm
TYPE	1 byte	Theo bảng 4.3
LEN	1 byte	0–64
PAYLOAD	LEN byte	Little-endian
CRC16	2 byte	CRC-16/CCITT-FALSE (poly 0x1021, init 0xFFFF) trên VER…PAYLOAD; CRC("123456789") = 0x29B1

Bảng 4.2 — Cấu trúc khung (115200 baud, 8N1)

Quy tắc: mọi khung (trừ ACK, NACK, PONG, STATUS) phải được ACK trong 200 ms (PING được trả bằng PONG, GET_STATUS bằng STATUS); quá hạn gửi lại tối đa 3 lần cùng SEQ. Bên nhận nhớ SEQ vừa xử lý; khung trùng SEQ chỉ ACK lại, không thực hiện lần hai. Khung sai CRC hoặc LEN > 64 bị bỏ và đếm lỗi; bộ giải mã tự dò lại SOF.

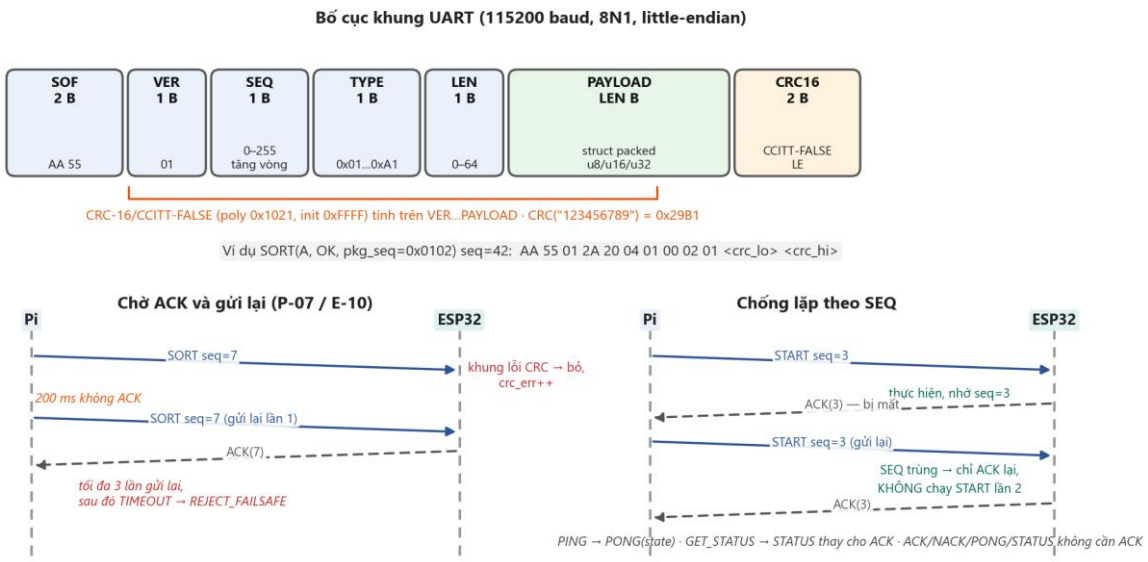
TYPE	Tên	Hướng	Payload
0x01	PING	Pi→ESP	—
0x10	START	Pi→ESP	speed u8 (30–100 %)
0x11	STOP	Pi→ESP	—
0x12	ESTOP	Pi→ESP	—
0x13	ESTOP_RESET	Pi→ESP	—
0x20	SORT	Pi→ESP	target u8 (1=A,2=B,3=REJECT), reason u8 (0=OK,1=DEFECT,2=SECURITY,3=FAILSAFE), pkg_seq u16
0x40	SET_PARAM	Pi→ESP	param_id u8, value u16
0x50	OTA_CHECK	Pi→ESP	—
0x60	GET_STATUS	Pi→ESP	—
0x80	ACK	ESP→Pi	acked_seq u8
0x81	NACK	ESP→Pi	acked_seq u8, err u8 (1=CRC,2=UNKNOWN_TYPE,3=BAD_STATE,4=BAD_PARAM,
0x82	PONG	ESP→Pi	state u8
0x90	EVT_PKG_DETECTED	ESP→Pi	pkg_seq u16
0x91	EVT_SORT_DONE	ESP→Pi	pkg_seq u16, target u8
0x92	EVT_ESTOP	ESP→Pi	source u8 (1=PHYSICAL, 2=REMOTE)
0x93	EVT_ESTOP_CLEARED	ESP→Pi	—
0x94	EVT_FAULT	ESP→Pi	code u8 (1=DECISION_TIMEOUT,2=LINK_LOST,3=SENSOR,4=WDT_R
0xA0	EVT_OTA_STATUS	ESP→Pi	stage u8, reason u8, progress u8, version_code u32
0xA1	STATUS	ESP→Pi	state u8, fw_version_code u32, uptime_s u32, pkg_count u16, crc_er reset_reason u8

Bảng 4.3 — Loại khung. Mã trạng thái: 0 BOOT, 1 IDLE, 2 RUNNING, 3 DETECTED, 4 SORTING, 5 ESTOP, 6 FAULT, 7 OTA

id	Tham số	Đơn vị	Miền	Mặc định
1	belt_speed	%	30–100	60
2	servo_a_home	độ	0–180	0

id	Tham số	Đơn vị	Miền	Mặc định
3	servo_a_divert	độ	0–180	45
4	servo_b_home	độ	0–180	0
5	servo_b_divert	độ	0–180	45
6	t_clear_a	ms	500–8000	2500
7	t_clear_b	ms	500–8000	3500
8	t_clear_reject	ms	500–10000	5000
9	t_rearm	ms	0–3000	500
10	t_decision_timeout	ms	1000–10000	3000
11	hcsr04_threshold	mm	20–400	80

Bảng 4.4 — Tham số SET_PARAM (ngoài danh sách hoặc ngoài miền → NACK BAD_PARAM); ESP32 lưu NVS



Hình 4.2 — Bố cục byte của khung UART, cơ chế chờ ACK/gửi lại và chống lặp theo SEQ

4.3 MQTT với ThingsBoard

Topic theo Device MQTT API v1: telemetry `v1/devices/me/telemetry`; client attributes `v1/devices/me/attributes`; nhận RPC `v1/devices/me/rpc/request/+`; trả RPC `v1/devices/me/rpc/response/{request_id}`.

Nhóm	Khóa	Kiểu	Khi gửi
Kiện hàng	pkg_id, warehouse, qr_status, ai_label, ai_conf, decision, reason, latency_ms (+ ai_defect_type khi lỗi)	string / float / int	Mỗi kiện
Ảnh lỗi	defect_image (JPEG 320×240 base64 ≤ 40 KB), defect_label	string	Chỉ khi REJECT_DEFECT
Bộ đếm	cnt_total, cnt_a, cnt_b, cnt_reject_defect, cnt_reject_security, cnt_reject_failsafe	int	Mỗi kiện và khi reset

Nhóm	Khóa	Kiểu	Khi gửi
Trạng thái	conveyor_state, esp_link	string / bool	Khi đổi và mỗi 10 s
Sức khỏe	cpu_temp, cpu_load, mem_used_pct, disk_used_pct	float	Mỗi 10 s
Bảo mật	security_alert, security_event_type, security_level, security_detail	bool / string	Khi có sự kiện
Audit RPC	rpc_log (JSON một dòng)	string	Mỗi RPC
Bất thường	anomaly_score, anomaly_flag, anomaly_top_features	float / bool / string	Mỗi 10 s
OTA	ota_stage, ota_reason, ota_progress	string / int	Khi ESP32 báo

Bảng 4.5 — Telemetry (đặc tả 4.3). Client attributes: fw_version (ESP32), sw_version (Pi), model_version, qr_format_version

4.4 Whitelist RPC

Phương thức	Tham số	Điều kiện	Hành động
start	{"speed": 30–100}	ESP32 ở IDLE	UART START
stop	{}	—	UART STOP
emergencyStop	{}	Luôn nhận, không tính rate limit	UART ESTOP
resetEstop	{}	ESP32 ở ESTOP hoặc FAULT	UART ESTOP_RESET
resetCounters	{}	—	Đặt bộ đếm về 0
resetUsedCodes	{"confirm": "RESET"}	device.demo_mode: true	Xóa bảng used_codes
otaCheck	{}	ESP32 ở IDLE	UART OTA_CHECK
getStatus	{}	—	Trả trạng thái hiện tại

Bảng 4.6 — RPC được phép. Trả về {"ok": true, ...} hoặc {"ok": false, "error": METHOD_NOT_ALLOWED | BAD_PARAMS | BAD_STATE | RATE_LIMITED | ESP_TIMEOUT}; giới hạn 5 lệnh/10 s

4.5 Manifest Secure OTA

```
{"created_at":"2026-11-02T09:15:00Z","device_model":"sorter-
esp32","fw_file":"firmware.bin","fw_sha256":"<64 hex>","fw_size":912384,"key_id":"<16
hex>","version_code":3,"version_name":"1.0.2"}
```

- manifest.json: một dòng, khóa sắp xếp, không khoảng trắng. manifest.sig: base64 của chữ ký ECDSA P-256/SHA-256 dạng DER ký trên đúng các byte của manifest.json.
- Báo cáo từ thiết bị: POST /ota/report với {device, version_code, result: SUCCESS|REJECTED|ROLLED_BACK, reason}.
- Stage: 1 CHECKING, 2 DOWNLOADING, 3 VERIFIED, 4 REJECTED, 5 REBOOTING, 6 SUCCESS, 7 ROLLED_BACK, 8 UP_TO_DATE. Reason: 0 NONE, 1 SIGNATURE_INVALID, 2 HASH_MISMATCH, 3

ROLLBACK_REJECTED, 4 MODEL_MISMATCH, 5 TLS_ERROR, 6 DOWNLOAD_ERROR, 7 SELFTEST_FAILED, 8 BAD_STATE.

4.6 File cấu hình của Pi

`/etc/sorter/config.yaml` gồm các mục device, camera (source: picamera2, resolution, settle_ms, max_qr_frames, roi, lock_after_s), qr (hmac_key_file, max_age_days, future_skew_s), ai (backend onnx|tflite|dummy, model_path, labels_path, model_card_path, input_size, min_confidence), uart, thingsboard (host, port 8883, ca_cert, client_cert, client_key, health_interval_s, image_max_kb), rpc.rate_limit, storage, anomaly. Lớp `SorterConfig` trong `pi/sorter/config.py` kiểm tra kiểu, miền giá trị, ROI nằm trong ảnh, sự tồn tại của file khóa/chứng chỉ và quyền 600; sai thì dừng ngay khi khởi động với thông điệp rõ ràng. Khóa bí mật không bao giờ nằm trong YAML.

5. Hiện thực chi tiết từng mô-đun

Mã nguồn được tổ chức thành monorepo `smart-sorter/` với 10 mô-đun theo đúng thứ tự đề xuất của đặc tả. Mỗi file ghi rõ mã chức năng mà nó hiện thực (ví dụ `# P-06`, `/* E-05 */`) để truy vết. Toàn bộ phần Python chạy được ở chế độ `--sim` trên laptop không cần phần cứng.

5.1 P1 — Khung repo và giao thức dùng chung

- `shared/protocol/protocol.yaml`: SOF, VER, LEN tối đa, 19 loại khung với payload và trạng thái được phép, mã NACK/fault/target/reason, stage/reason OTA, 11 tham số SET_PARAM.
- `shared/protocol/gen.py` sinh `esp32/components/protocol/include/protocol_gen.h` (enum, struct packed, `_Static_assert` kích thước, bảng tham số, hàm `proto_expected_len`, `proto_allowed_states`) và `pi/sorter/protocol_gen.py` (IntEnum, dataclass Payload với `pack()/unpack()` bằng `struct` little-endian, `validate_fields`).
- `pi/sorter/uart_codec.py`: `crc16_ccitt_false` (bảng tra), `encode_frame`, `FrameDecoder` nhận dữ liệu theo mảnh bất kỳ, tự dò lại SOF, bỏ khung sai CRC/LEN và đếm thống kê. `esp32/components/protocol/protocol.c` là bản C cùng thuật toán, không phụ thuộc ESP-IDF nên test được trên máy host.
- `shared/protocol/test_vectors.json`: 14 vector (khung không payload, SET_PARAM với SEQ 255, 2 khung dính liền, rác trước SOF, sai CRC, LEN = 65, cắt đôi qua hai lần feed, feed từng byte, payload 64 byte, sai VER, AA 55 trong payload). `pi/tests/test_uart_codec.py` (22 test) và `host_test.c` cùng đọc file này.

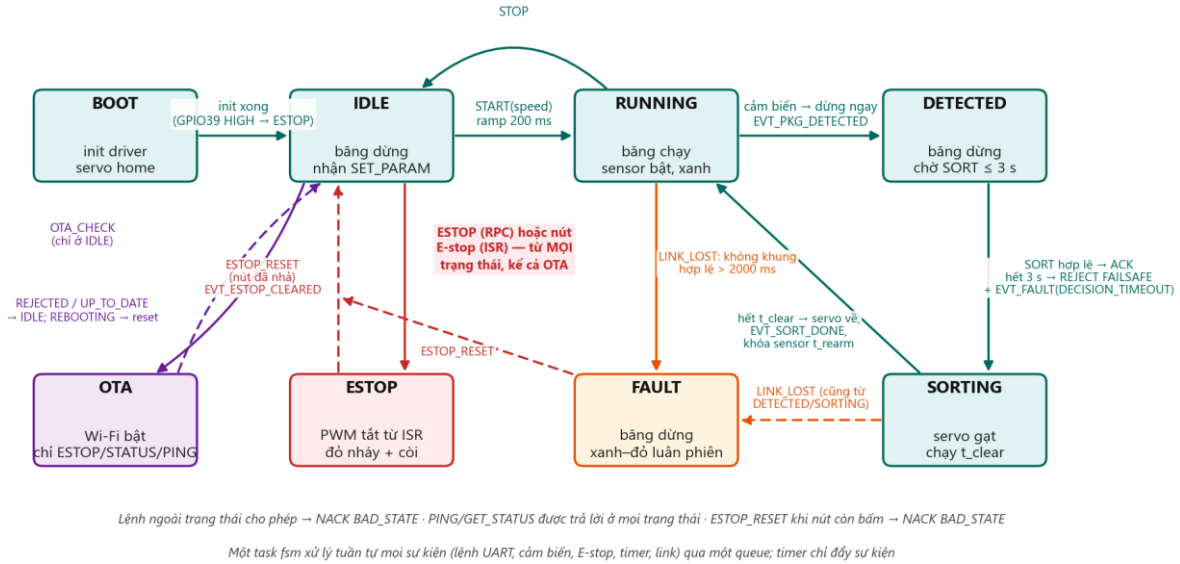
Kết quả: 22 test Python pass; CRC của chuỗi "123456789" bằng 0x29B1 đúng chuẩn. Test C đã viết nhưng máy phát triển không có gcc nên cần biên dịch bằng `make test-c` trên máy khác.

5.2 P2 — Firmware ESP32 phần chấp hành

Firmware viết bằng C trên ESP-IDF v5.x, tách thành 10 component: `board` (định nghĩa chân), `motor` (L298N, LEDC 1 kHz 10 bit, ramp mềm 200 ms bằng esp_timer, hàm `motor_stop_from_isr` thuần thanh ghi), `servo` (LEDC 50 Hz 14 bit, góc 0–180 → xung 500–2500 µs), `sensor` (TCRT5000 ngắt + chống dội 20 ms, hoặc HC-SR04 đo mỗi 30 ms xác nhận 2 lần, chọn bằng Kconfig), `indicator` (bảng pattern LED/còi chạy bằng một timer 50 ms, không chặn), `estop` (GPIO39 ngắt hai cạnh, ISR tắt PWM ngay), `link` (UART2, bộ giải mã, ACK/NACK, gửi lại, chống lặp SEQ, STATUS mỗi 1 s, watchdog liên kết 2000 ms, Task Watchdog), `params` (NVS + whitelist min/max từ `protocol_gen.h`), `fsm` (máy trạng thái) và `ota_secure` (mục 5.8).

Trạng thái	Sự kiện	Hành động	Sang
IDLE	START(30–100)	ACK, motor ramp 200 ms, sensor on, LED xanh	RUNNING
RUNNING	cảm biến	motor_stop ngay, pkg_seq++, EVT_PKG_DETECTED, timer t_decision	DETECTED
DETECTED	SORT(target, reason, pkg_seq)	sai pkg_seq → NACK BAD_PARAM; đúng → ACK, servo A/B gạt (REJECT không gạt), motor chạy, timer t_clear	SORTING
DETECTED	hết t_decision (3 s)	như SORT(REJECT, FAILSAFE) + EVT_FAULT(DECISION_TIMEOUT)	SORTING
SORTING	hết t_clear	servo home, LED/còi theo reason, EVT_SORT_DONE, sensor khóa t_rearm	RUNNING
RUNNING/DETECTED/SORTING	không khung hợp lệ > 2000 ms	dừng băng, FAULT; EVT_FAULT(LINK_LOST) gửi khi link trở lại	FAULT
mọi (≠ ESTOP)	lệnh ESTOP hoặc nút E-stop	ISR tắt PWM, servo home, hủy timer, đỏ nháy + còi, EVT_ESTOP(src)	ESTOP
ESTOP / FAULT	ESTOP_RESET	nút còn bấm → NACK BAD_STATE; đã nhả → ACK, EVT_ESTOP_CLEARED	IDLE
IDLE	OTA_CHECK	ota_secure_start, xanh nháy nhanh; chỉ nhận ESTOP/GET_STATUS/PING	OTA
mọi	lệnh ngoài proto_allowed_states	NACK BAD_STATE	giữ

Bảng 5.1 — Bảng chuyển trạng thái (đầy đủ trong đầu file esp32/components/fsm/fsm.c)

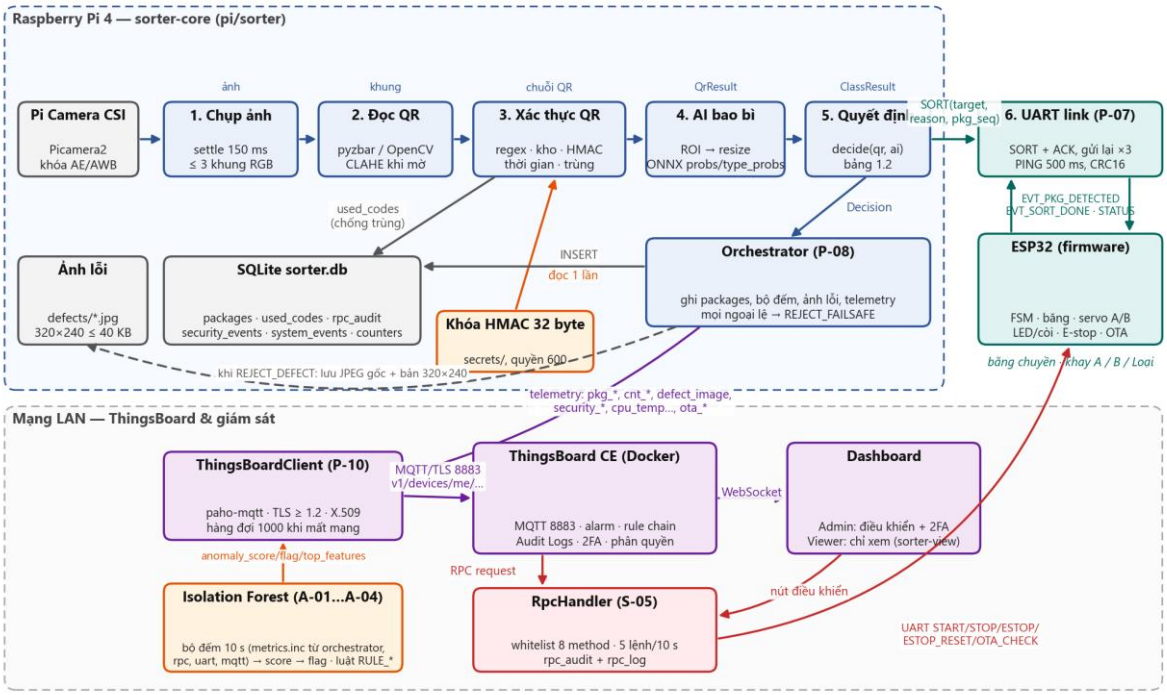


Hình 5.1 — Máy trạng thái của firmware ESP32 (E-01…E-05, E-08, E-09)

- Mọi sự kiện (lệnh UART, cảm biến, E-stop, timer, link) đi vào một queue, một task fsm xử lý tuần tự nên không có điều kiện tranh chấp; timer chỉ đẩy sự kiện, không gọi driver.
- Chế độ kiểm tra đấu dây (Kconfig `SORTER_HW_TEST`): quay động cơ 2 s, quét servo A rồi B, nháy từng LED, kêu còi, in trạng thái cảm biến và E-stop mỗi 200 ms.
- `partitions.csv` bố cục OTA cho flash 4 MB (nvs, otadata, phy_init, ota_0 và ota_1 mỗi phân vùng 0x1E0000); `sdkconfig.defaults` bật `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`, Task Watchdog 5 s, FreeRTOS 1000 Hz, tắt HTTP không mã hóa cho OTA.
- Điểm lệch có chủ đích: khóa NVS của tham số là `p<id>` vì NVS giới hạn tên khóa 15 ký tự (`t_decision_timeout` dài 18).

Máy phát triển không có ESP-IDF nên firmware chưa được biên dịch; các API cần đối chiếu theo phiên bản IDF được đánh dấu trong `esp32/README.md` (gpio_ll trong ISR, esp_timer_start_once từ ISR, esp_tls error handle, mbedTLS 3.x).

5.3 P3 — sorter-core trên Raspberry Pi



Hình 5.2 — Luồng dữ liệu của sorter-core: pipeline 6 bước, lưu trữ, telemetry MQTT, RPC và bộ đếm cho phát hiện bất thường

Package `pi/sorter/` là phần mềm điều phối chính, khoảng 6 600 dòng Python có type hints, dataclass và logging JSON. Các thành phần:

File	Mã	Nội dung
config.py	P-01	Dataclass cấu hình lồng nhau, kiểm tra miền, đường dẫn, quyền file khóa 600; lỗi rõ ràng khi khởi động
camera.py	P-02	`Picamera2Camera`: camera CSI qua picamera2, `create_still_configuration` RGB888, mở một lần, sau 2 s đọc metadata rồi khóa AeEnable/AwbEnable với ExposureTime, AnalogueGain, ColourGains; `FolderCamera` cho sim
qr_reader.py	P-03	pyzbar, dự phòng cv2.QRCodeDetector; thử tối đa 3 khung, tăng tương phản CLAHE; > 1 mã → MULTI_QR
qr_security.py	S-03, P-04	Parse + regex, HMAC-SHA256 base64url, `hmac.compare_digest`, kiểm tra FUTURE/EXPIRED, chống trùng bằng INSERT nguyên tử vào used_codes; `load_hmac_key` không bao giờ log khóa
decision.py	P-06	Hàm thuần `decide(qr, ai)` → (target, code, reason) đúng bảng 2.1; unit test mọi hàng
uart_link.py	P-07	Thread đọc + thread PING, `send_command` chờ ACK và gửi lại cùng SEQ, hàng đợi sự kiện, tự ACK sự kiện của ESP32, chống lặp 16 SEQ gần nhất, trạng thái liên kết, đếm lỗi CRC
fake_esp32.py	P-14	Giả lập đầy đủ máy trạng thái E-01 (gồm decision timeout, link lost, E-stop, chống lặp SEQ, NACK) qua `LoopbackTransport` trong bộ nhớ; có `trigger_sensor`, `press_estop`, `drop_next_acks`, `inject_bad_crc`

File	Mã	Nội dung
storage.py	P-09	SQLite WAL, bảng packages, used_codes, rpc_audit, security_events, system_events, counters, schema_version
counters.py / health.py	P-11 / P-12	6 bộ đếm bền, khôi phục sau khởi động; nhiệt độ CPU từ /sys/class/thermal, psutil
security_events.py	S-09	`SecurityLog.emit` ghi DB + telemetry security_* + log
defect_images.py	P-09	Lưu JPEG gốc, bản 320×240 giảm chất lượng dần tới ≤ 40 KB base64, dọn ảnh cũ khi quá 500
orchestrator.py	P-08	Vòng chính: EVT_PKG_DETECTED → pipeline (settle, chụp, QR, verify, AI, decide, SORT) với đo thời gian từng bước; xử lý EVT_ESTOP/FAULT/OTA_STATUS/STATUS; thread sức khỏe 10 s; mọi ngoại lệ → REJECT_FAILSAFE
factories.py / main.py	P-14, P-15	Lắp ráp camera/classifier/telemetry/uart/RPC/anomaly bằng import lười theo config; CLI `--config --sim --log-level`; SIGTERM dừng sạch
logging_setup.py	P-13	Console + file xoay vòng 5 × 5 MB `sorter.jsonl`, mỗi dòng một JSON

Bảng 5.2 — Thành phần sorter-core

Công cụ kèm theo (`pi/tools/`): `make_hmac_key.py` (sinh khóa 32 byte, chmod 600), `qr_generator.py` (PNG từng mã + trang A4 PDF lưới 3×4 có nhãn, cờ `--forge` tạo QR chữ ký sai), `capture_dataset.py` (chụp tại trạm bằng Picamera2, gán nhãn phím 1–5, chế độ `--auto` chụp khi có EVT_PKG_DETECTED), `uart_monitor.py`, `esp_bench.py` (16 ca kiểm tra ESP32 thật hoặc `--fake`). Dịch vụ `systemd/sorter.service` chạy bằng user `sorter` với NoNewPrivileges, ProtectSystem=strict; `scripts/install_pi.sh` tạo user, thư mục, venv dùng `python3-picamera2` từ apt, hướng dẫn bật UART.



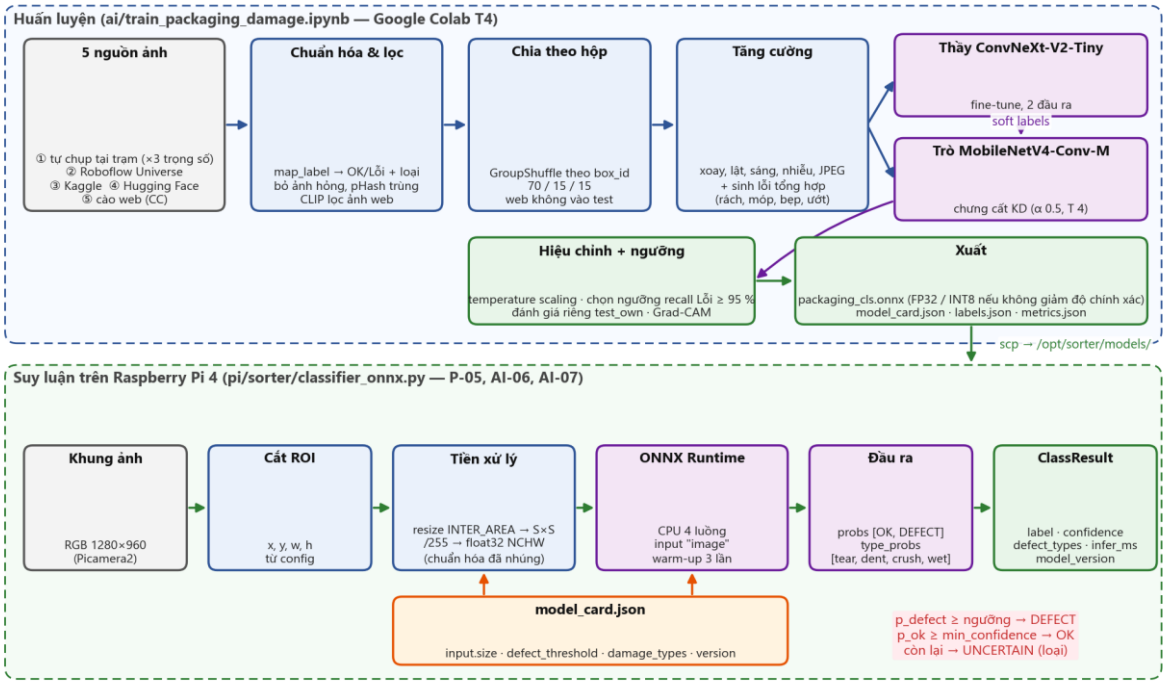
Hình 5.3 — Ảnh mẫu do tests/fixtures/make_images.py sinh: QR hợp lệ kho A, kho B, hộp rách + QR hợp lệ, QR giả (chữ ký sai)

Chạy `python -m sorter.main --sim --config config.sim.yaml` trên laptop cho kết quả đúng với cả 8 ảnh mẫu (bảng 7.2).

5.4 P4 — AI kiểm tra bao bì

Nhóm chọn **phân loại ảnh** thay vì object detection vì camera cố định và mỗi lần chỉ có một hộp trong ROI. Notebook chính thức `ai/train_packaging_damage.ipynb` thực hiện trọn quy trình: thu dữ liệu từ 5 nguồn (ảnh tự chụp tại trạm là quan trọng nhất, Roboflow Universe, Kaggle, Hugging Face, cào web có giấy phép CC), chuẩn hóa nhãn về hệ chung

(OK/Lỗi + loại lỗi tear/dent/crush/wet), khử trùng lặp bằng perceptual hash, lọc ảnh web bằng CLIP, chia tập theo hộp (ảnh cùng một hộp không nằm ở cả train và test), tăng cường dữ liệu và sinh lỗi tổng hợp, train mô hình thầy ConvNeXt-V2-Tiny rồi chưng cất sang MobileNetV4-Conv-Medium (2 đầu ra), hiệu chỉnh xác suất, chọn ngưỡng đạt recall lớp Lỗi ≥ 95 %, xuất ONNX FP32 + INT8 và `model_card.json`.



Hình 5.4 — Pipeline AI: huấn luyện trên Colab (5 nguồn ảnh, thầy ConvNeXt-V2 → trò MobileNetV4, xuất ONNX) và suy luận trên Pi

- `pi/sorter/classifier_onnx.py` (P-05, AI-06, AI-07): `OnnxClassifier` dùng onnxruntime CPU 4 luồng, đọc `model_card.json` để lấy kích thước đầu vào, tên tensor, `defect_threshold`, loại lỗi và phiên bản; tiền xử lý cắt ROI → resize INTER_AREA → float32/255 → NCHW (chuẩn hóa đã nhúng trong model). Hỗ trợ cả model 2 đầu ra của notebook và model 1 đầu ra kiểu YOLOv8-cls với labels.json. Quy tắc an toàn: p_defect ≥ ngưỡng → DEFECT; p_ok ≥ min_confidence → OK; còn lại UNCERTAIN (bị loại). `TfliteClassifier` tương tự với tflite_runtime và input int8.
- `ai/prepare_dataset.py` (AI-02): loại ảnh hỏng, thống kê theo lớp/hộp, chia 70/15/15 theo box_id, cảnh báo lệch lớp > 1,5 lần, xuất cấu trúc ultralytics. `ai/train_yolo_cls.py` + notebook (AI-03/04): YOLOv8n-cls 224, 50 epoch, patience 10, export ONNX + labels + model_card. `ai/train_mobilenetv2.py`: phương án Keras + TFLite float16/int8.
- `ai/evaluate.py` (AI-05/07): accuracy, precision/recall/F1 từng lớp, confusion_matrix.png, lưới ảnh sai, quét ngưỡng trên tập val chọn ngưỡng đạt recall DEFECT ≥ 95 % và ghi vào model_card. `pi/tools/benchmark_model.py`: warm-up 10, đo 100 lần, mean/p50/p95/max và RAM, ghi benchmark.json.

- `ai/export_dummy_model.py` tạo model ONNX nhỏ (681 byte) đúng định dạng notebook để test và chạy sim với backend onnx khi chưa có model thật.

Model thật chưa được train vì cần dataset chụp tại trạm (≥ 300 ảnh/lớp). Các số đo AI trong chương 7 là với model dummy, chỉ chứng minh đường đi dữ liệu.

5.5 P5 — ThingsBoard, MQTT TLS, RPC và dashboard

- `thingsboard/pki/`: `make_ca.sh` (CA EC P-256, 5 năm), `make_server_cert.sh` (SAN DNS + IP, 2 năm, serverAuth), `make_device_cert.sh` (CN = tên thiết bị, 1 năm, clientAuth, khóa 600), `make_ota_server_cert.sh`. Script idempotent, không ghi đè khóa khi thiếu `--force`, in hướng dẫn copy file đi đâu.
- `thingsboard/docker-compose.yml`: image `thingsboard/tb-postgres:3.7.0`, MQTT_SSL_ENABLED, PEM cert/key mount read-only, chỉ công bố cổng web và 8883, không công bố 1883; DEFAULT_INACTIVITY_TIMEOUT 30 s; khóa tài khoản sau 5 lần sai.
- `pi/sorter/tb_client.py` (T-02…T-04, S-01, S-04): paho-mqtt 2.x, `tls_set` với CA + cert/key thiết bị, TLS ≥ 1.2, kiểm tra hostname; không có bất kỳ tùy chọn bỏ xác minh nào. Kết nối lại với backoff 1–60 s và đếm số lần (đặc trưng cho A-01); hàng đợi 1000 bản tin khi mất mạng; subscribe RPC và trả response đúng topic; RPC chạy trong thread riêng để không chặn vòng mạng; bản tin > 48 KB bị từ chối.
- `pi/sorter/rpc_handler.py` (S-05, T-04): whitelist đúng bảng 4.6, schema kiểm tra kiểu/miền/khóa lạ, token bucket 5 lệnh/10 s, `emergencyStop` xử lý trước mọi thứ, kiểm tra trạng thái ESP32, ánh xạ lỗi UART sang ESP_TIMEOUT/BAD_STATE, ghi `rpc_audit` và gửi `rpc_log`; phương thức lạ → METHOD_NOT_ALLOWED + sự kiện bảo mật RPC_REJECTED.
- `thingsboard/widgets/`: thẻ ảnh hàng lỗi (base64 → img, nhãn, độ tin cậy, thời gian) và bảng điều khiển (nút gửi two-way RPC timeout 10 s, tự thêm requestedBy = email người đăng nhập; DỪNG KHẨN CẤP đỏ lớn gửi ngay; Reset E-stop và Reset mã đã dùng có xác nhận).
- `thingsboard/SETUP.md`: hướng dẫn từng bước device profile "Sorter" (inactivity 30 s, alarm Security alert / Anomaly / High CPU temperature), device sorter-pi-01 với credentials X.509, thiết bị gương sorter-view, rule chain sao chép telemetry, người dùng Admin (Tenant Administrator, 2FA TOTP cưỡng chế) và Viewer (Customer User chỉ thấy dashboard Giám sát), danh sách widget cho hai dashboard, lệnh nghiệm thu bằng openssl s_client và nmap. `thingsboard/tools/rpc_cli.py` gọi RPC qua REST để test whitelist.

5.6 P6 — Hardening Raspberry Pi và bảo vệ ESP32

- `pi/scripts/harden_pi.sh` (S-07): idempotent, hỏi xác nhận trước bước có thể tự khóa; kiểm tra `authorized_keys` trước khi tắt mật khẩu SSH; `sshd -t` trước khi reload; ufw default deny + chỉ 22/tcp từ subnet LAN; fail2ban (backend systemd

cho Bookworm); quyền 700/600 cho secrets và certs; tắt bluetooth/avahi/cups (hỏi trước); unattended-upgrades không tự reboot.

- `pi/scripts/security_audit.sh`: 28 mục PASS/FAIL/SKIP kèm lệnh kiểm chứng, xuất JSON, exit 1 nếu có FAIL — dùng để chụp slide.
- `esp32/SECURITY.md` (S-08): cảnh báo eFuse vĩnh viễn, kiểm tra chip revision, sinh khóa Secure Boot V2 RSA-3072, Flash Encryption chế độ Development, NVS encryption, tắt JTAG, thứ tự nạp lần đầu và nạp lại, cách chứng minh khi demo.
- `docs/security_slide.md` và `docs/threat_model.md`: bảng S-01→S-09 (mối đe dọa → biện pháp → cách chứng minh → file liên quan) và STRIDE cho 5 tầng, kèm mục Giới hạn trung thực.

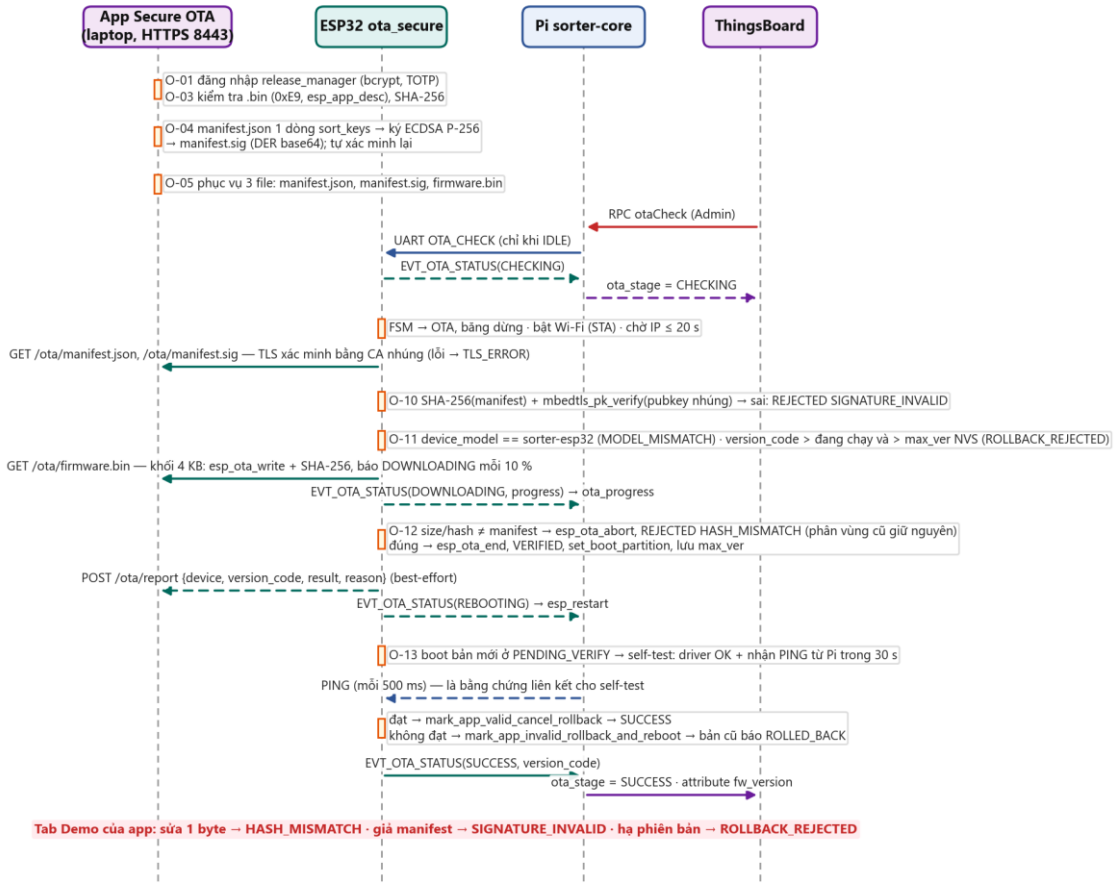
5.7 P7 — Ứng dụng Secure OTA trên máy tính

Ứng dụng `ota-app/` (Python 3.12 + PySide6, khoảng 3 900 dòng) tách logic khỏi giao diện nên test được không cần Qt:

- O-01 `auth.py`: tài khoản SQLite, bcrypt cost 12, mật khẩu ≥ 10 ký tự, khóa 5 phút sau 5 lần sai (lưu DB, vẫn khóa sau khi mở lại app), TOTP tùy chọn với QR để quét, phiên hết hạn sau 15 phút không thao tác, vai trò release_manager/viewer.
- O-02 `keys.py`: cặp khóa ECDSA P-256, private key PEM mã hóa bằng passphrase quyền 600, không bao giờ hiển thị; xuất `ota_pubkey.pem` và `esp32/components/ota_secure/include/ota_pubkey.h`; key_id = 8 byte đầu SHA-256 của public key DER.
- O-03 `firmware.py`: kiểm tra byte đầu 0xE9, đọc `esp_app_desc_t` tại offset 32 (magic 0xABCD5432) để lấy phiên bản, SHA-256, heuristic khối chữ ký Secure Boot V2.
- O-04 `manifest.py`: manifest một dòng sort_keys, ký DER base64, `verify_release_dir` kiểm tra đúng thứ tự như ESP32 (chữ ký → model → version → size → hash) và tự xác minh lại trước khi phát hành.
- O-05 `server.py`: aiohttp HTTPS cổng 8443 (TLS ≥ 1.2, cert do CA nội bộ ký) trong thread riêng, đúng 4 route, POST /ota/report ≤ 2 KB kiểm tra trường, mọi route khác 404, log mọi request.
- O-06 `history.py`: bảng releases, device_reports, request_log; xuất CSV. O-07 `demo_attacks.py`: sửa 1 bit sau header, giả manifest giữ chữ ký cũ, hạ phiên bản ký hợp lệ; nút khôi phục bản hợp lệ. `tools/verify_cli.py` xác minh ngoài giao diện.

Kết quả: 31 test pass, gồm server HTTPS thật với CA tự tạo trong test (client không tin CA và TLS ≤ 1.1 bị từ chối). Giao diện đã chạy thử headless đủ luồng tạo admin → đăng nhập → tạo khóa → ký và phát hành → demo tấn công → hết phiên.

5.8 P8 — Secure OTA phía ESP32



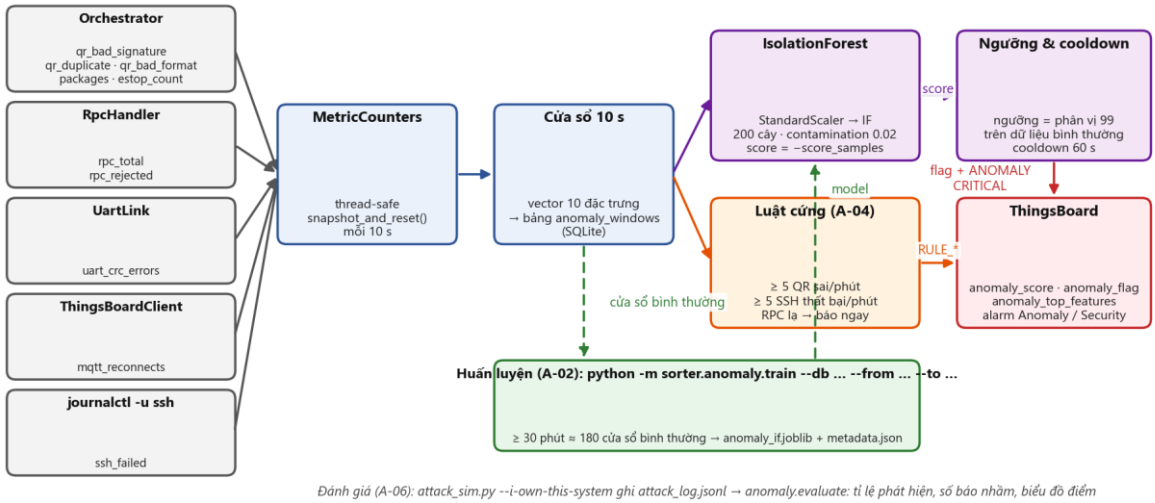
Hình 5.5 — Sơ đồ tuần tự Secure OTA: app ký và phục vụ HTTPS, ESP32 tải, xác minh ECDSA, chống rollback, kiểm tra SHA-256, tự kiểm tra và quay lui

- Component `esp32/components/ota_secure/` nhúng CA máy chủ OTA (EMBED_TXTFILES) và public key ECDSA; SSID/mật khẩu Wi-Fi và URL máy chủ lưu NVS namespace `ota_cfg` (nạp bằng `tools/provision_nvs.sh`), Wi-Fi chỉ bật trong OTA.
- Luồng (task riêng, stack 10 KB): CHECKING → tải manifest.json (≤ 1 KB) và manifest.sig (≤ 256 B) bằng esp_http_client với `cert_pem` = CA, không bao giờ bỏ xác minh → mbedTLS SHA-256 + `mbedtls_pk_verify` trên đúng byte manifest (sai → SIGNATURE_INVALID) → cJSON kiểm tra device_model (MODEL_MISMATCH), version_code > bản đang chạy và > max_ver trong NVS (ROLLBACK_REJECTED), fw_size ≤ phân vùng → `esp_ota_begin`, tải theo khối 4 KB vừa ghi vừa băm, báo DOWNLOADING mỗi 10 % → sai size/hash → `esp_ota_abort`, HASH_MISMATCH → `esp_ota_end`, `esp_ota_set_boot_partition`, lưu max_ver, báo cáo, REBOOTING.
- O-13: sau khởi động, nếu phân vùng ở trạng thái PENDING_VERIFY thì self-test (driver init + nhận PING hợp lệ từ Pi trong 30 s); đạt → `esp_ota_mark_app_valid_cancel_rollback`, không đạt → `esp_ota_mark_app_invalid_rollback_and_reboot`; bản cũ khi chạy lại báo ROLLED_BACK một lần. O-14: EVT_OTA_STATUS qua UART ở mọi stage,

Pi đẩy lên telemetry ota_stage/ota_reason/ota_progress và cập nhật attribute fw_version; POST /ota/report best-effort.

- `esp32/OTA_TEST.md` mô tả 4 ca (hợp lệ, sửa 1 byte, giả manifest, hạ phiên bản) và cách thử rollback bằng Kconfig `SORTER_OTA_SELFTEST_FAIL_DEMO`.

5.9 P9 — Phát hiện truy cập bất thường



Hình 5.6 — Pipeline phát hiện bất thường: bộ đếm từ các module → cửa sổ 10 s → Isolation Forest + luật cứng → alarm ThingsBoard

- A-01 `anomaly/features.py`: 10 đặc trưng mỗi cửa sổ 10 s (rpc_total, rpc_rejected, qr_bad_signature, qr_duplicate, qr_bad_format, uart_crc_errors, mqtt_reconnects, ssh_failed từ journalctl, estop_count, packages) lấy từ bộ đếm `MetricCounters` mà orchestrator, rpc_handler, uart_link và tb_client tăng; lưu bảng anomaly_windows.
- A-02 `anomaly/train.py`: StandardScaler + IsolationForest (200 cây, contamination 0,02, random_state 42) trên ≥ 180 cửa sổ bình thường; ngưỡng = phân vị 99; lưu joblib + metadata.
- A-03 `anomaly/detector.py`: thread 10 s, điểm = −score_samples, gửi anomaly_score/anomaly_flag/anomaly_top_features (3 đặc trưng có z-score lớn nhất), sự kiện ANOMALY mức CRITICAL với cooldown 60 s; chạy được khi chưa có model (chỉ thu dữ liệu + luật cứng).
- A-04 `anomaly/rules.py`: ≥ 5 QR chữ ký sai/phút, ≥ 5 SSH thất bại/phút, RPC lạ → báo ngay. A-05 `tools/attack_sim.py` bắt buộc cờ `--i-own-this-system`, kịch bản rpc/ssh/qr/uart, ghi attack_log.jsonl. A-06 `anomaly/evaluate.py`: tỉ lệ phát hiện từng kịch bản và số báo nhầm, bảng Markdown + biểu đồ.

Kết quả: 10 test với dữ liệu tổng hợp pass (cửa sổ tấn công vượt ngưỡng, ~100 % cửa sổ bình thường dưới ngưỡng, luật kích hoạt đúng một lần trong cooldown).

5.10 P10 — Tích hợp, kiểm thử demo, số liệu

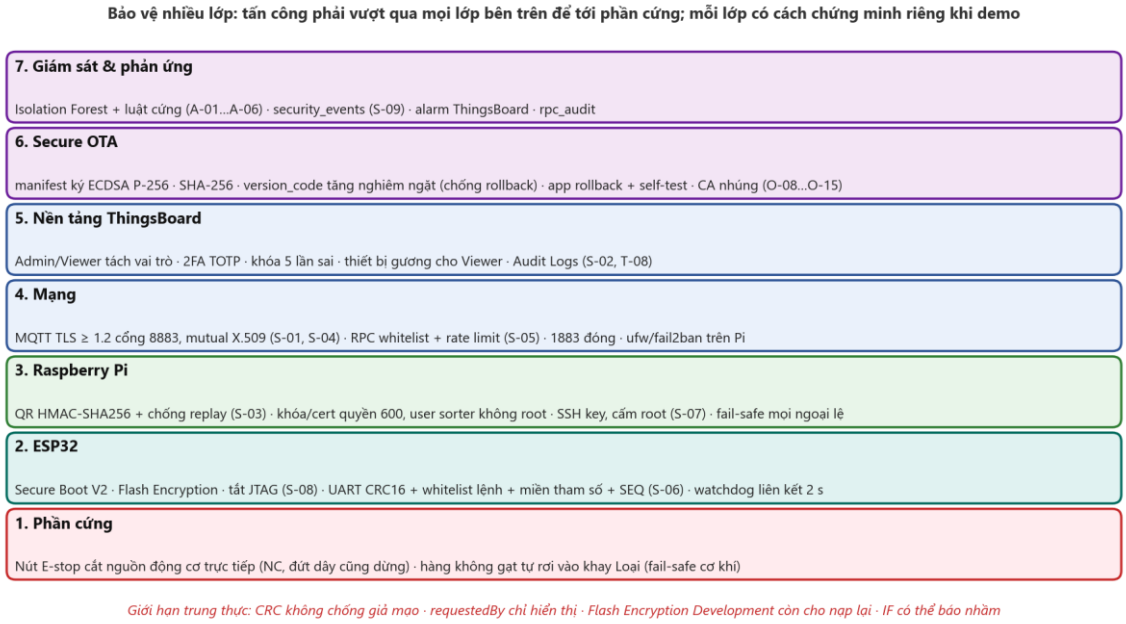
- `docs/RUNBOOK.md`: thứ tự bật hệ thống, checklist trước demo, xử lý sự cố thường gặp (8883, UART, ảnh, AI, QR, E-stop, OTA).
- `pi/tools/demo_check.py`: chạy tự động các kịch bản 5.1/5.2 ở chế độ sim với ESP32 giả lập (và chế độ thật hướng dẫn từng bước), in PASS/FAIL. `pi/tools/report_metrics.py`: từ SQLite và các file kết quả sinh `docs/metrics.md` dạng bảng cho slide.
- `docs/traceability.md`: bảng truy vết đủ 75 mã chức năng → file → test/kịch bản → trạng thái. `docs/demo_checklist.md`: checklist in giấy cho buổi demo.

6. Bảo mật tổng hợp

Mã	Mối đe dọa	Biện pháp	Chứng minh khi demo	File liên quan
S-01	Thiết bị giả kết nối lên nền tảng; nghe lén MQTT	X.509 mutual TLS: CA nội bộ EC P-256, cert server cho ThingsBoard, cert thiết bị CN = sorter-pi-01 hạn 1 năm; không dùng access token	Kết nối không cert hoặc cert lạ bị từ chối; openssl s_client	thingsboard/pki, tb_client.py
S-02	Chiếm tài khoản dashboard	Admin/Viewer, 2FA TOTP cưỡng chế Admin, chính sách mật khẩu, khóa sau 5 lần sai	Đăng nhập Viewer không có nút điều khiển; Admin phải nhập mã 2FA	thingsboard/SETUP.md
S-03	QR giả, QR in lại	HMAC-SHA256 khóa 32 byte, so sánh hằng thời gian, giới hạn thời gian ±, bảng used_codes chống replay; khóa quyền 600, không log	QR giả → alarm BAD_SIGNATURE; quét lại → DUPLICATE	qr_security.py, make_hmac_key.py
S-04	Hạ cấp/giả mạo kênh MQTT	TLS ≥ 1.2 cổng 8883, luôn xác minh server, code không có tùy chọn bỏ xác minh; 1883 không mở	nmap -p 1883 đóng; test khẳng định không gọi tls_insecure_set	tb_client.py, docker-compose.yml
S-05	Lệnh điều khiển trái phép	Whitelist 8 phương thức, schema tham số, 5 lệnh/10 s, audit rpc_audit + rpc_log, RPC_REJECTED	rpc_cli gọi phương thức lạ → METHOD_NOT_ALLOWED + alarm	rpc_handler.py
S-06	Lỗi/nhiễu đường UART	CRC16, whitelist lệnh và miền tham số trên ESP32, SEQ chống lặp (CRC chống lỗi, không chống giả mạo; UART là dây nội bộ)	esp_bench: khung sai CRC bị bỏ, crc_err tăng	protocol.c, uart_codec.py, link.c
S-07	Xâm nhập Pi qua mạng	SSH chỉ key, cấm root, ufw chỉ 22 từ LAN,	security_audit.sh PASS toàn bộ	pi/scripts

Mã	Mối đe dọa	Biện pháp	Chứng minh khi demo	File liên quan
		fail2ban, user sorter không root, file 600		
S-08	Đọc/ghi firmware ESP32	Secure Boot V2, Flash Encryption, tắt JTAG (eFuse, thử trên board dự phòng)	Nạp firmware không ký bị từ chối; read_flash ra dữ liệu mã hóa	esp32/SECURITY.md
S-09	Không có dấu vết tấn công	Mọi sự kiện có loại, mức, chi tiết, thời gian; lưu DB và gửi ThingsBoard	Bảng alarm bảo mật trên dashboard	security_events.py
O-xx	Firmware độc hại / rollback	Manifest ký ECDSA P-256, SHA-256, version_code tăng nghiêm ngặt, app rollback tự quay lui	3 bản demo đều bị từ chối đúng lý do	ota-app, ota_secure.c
A-xx	Tổ hợp bất thường không lường trước	Isolation Forest + luật cứng	attack_sim → alarm Anomaly ≤ 20 s	pi/sorter/anomaly

Bảng 6.1 — Tổng hợp biện pháp bảo mật



Hình 6.1 — Bảy lớp bảo vệ từ phần cứng đến giám sát và giới hạn được nêu trung thực

Giới hạn được nêu trung thực

- CRC16 trên UART chống lỗi đường truyền, không chống giả mạo; chấp nhận được vì UART là dây nội bộ trong hộp máy.
- `requestedBy` trong RPC chỉ để hiển thị; nguồn tin cậy về người gửi là Audit Logs của ThingsBoard.

- Flash Encryption chế độ Development vẫn cho nạp lại qua UART; chế độ Release mới khóa hoàn toàn.
- Isolation Forest có thể báo nhầm; ngưỡng phân vị 99 và cooldown 60 s giảm nhưng không loại bỏ hoàn toàn.
- Nút E-stop vật lý cắt nguồn động cơ bằng phần cứng; phần mềm chỉ là lớp bảo vệ thứ hai.

7. Kiểm thử và kết quả

7.1 Bộ kiểm thử tự động

Bộ test	Số test	Phạm vi
pi/tests/test_uart_codec.py	22	CRC chuẩn, bố cục khung, 14 test vector dùng chung với C, pack/unpack mọi payload
pi/tests/test_decision.py, test_qr_security.py	—	Mọi hàng bảng luật; mọi mã xác thực QR gồm replay, sửa 1 ký tự chữ ký
pi/tests/test_uart_link.py	—	ACK, NACK BAD_STATE/BAD_PARAM, gửi lại khi mất ACK, SEQ lặp, mất link, decision timeout
pi/tests/test_orchestrator.py	5	End-to-end sim với ảnh mẫu: A, B, rách, QR giả, không QR, trùng
pi/tests/test_storage.py, test_config.py	—	used_codes nguyên tử, bộ đếm bền, lỗi cấu hình
pi/tests/test_classifier_onnx.py	13	OnnxClassifier với model dummy: OK/DEFECT/UNCERTAIN, ROI, ngưỡng từ model_card, model generic
pi/tests/test_rpc_handler.py	57	Mọi nhánh whitelist, BAD_PARAMS, BAD_STATE, RATE_LIMITED, METHOD_NOT_ALLOWED, ESP_TIMEOUT, audit
pi/tests/test_tb_client.py	17	Mock paho: TLS 3 file, không bao giờ tls_insecure, topic/payload, hàng đợi, RPC response
pi/tests/test_anomaly.py	10	Train/score tổng hợp, luật cứng, store, detector, attack_sim thiếu cờ
pi/tests/test_demo_check.py	3	Kịch bản demo 1, 2, 3, 4, 7 ở chế độ sim qua App thật + FakeEsp32
Tổng pi/	203	python -m pytest → 203 passed trong ~29 s
ota-app/tests	31	Ký/xác minh, 3 kiểu tấn công, chặn version, khóa tài khoản, firmware giả, HTTPS thật

Bảng 7.1 — Kiểm thử tự động (Python 3.12, Windows 11)

7.2 Chạy mô phỏng toàn pipeline

#	Ảnh	pkg_id	qr_status	ai_label	Quyết định	Latency (ms)
1	01_ok_A.jpg	PKG000001	VALID	OK	SORT_A	91
2	02_ok_B.jpg	PKG000002	VALID	OK	SORT_B	87
3	03_defect_tear_A.jpg	PKG000003	VALID	DEFECT	REJECT_DEFECT	85
4	04_forged_A.jpg	PKG000004	BAD_SIGNATURE	—	REJECT_SECURITY	81
5	05_noqr.jpg	—	NO_QR	—	REJECT_SECURITY	618
6	06_expired_A.jpg	PKG000006	EXPIRED	—	REJECT_SECURITY	110
7	07_dup_A.jpg	PKG000001	DUPLICATE	—	REJECT_SECURITY	102
8	08_uncertain_A.jpg	PKG000008	VALID	UNCERTAIN	REJECT_DEFECT	81

Bảng 7.2 — Kết quả `python -m sorter.main --sim` (DummyClassifier, ESP32 giả lập); bộ đếm cuối: total 8, A 1, B 1, reject_defect 2, reject_security 4, failsafe 0

Khi đổi `ai.backend` sang `onnx` với model dummy và bật anomaly, pipeline vẫn chạy đúng: OnnxClassifier nạp model_card (version 0.0.0-dummy, 224×224, ngưỡng 0,6), attribute model_version được gửi, detector thu cửa sổ 1 s và báo "chưa có model". Thời gian từng bước đo được trên laptop: chụp 10–25 ms, đọc QR 58–100 ms (pyzbar), xác thực < 1 ms, AI 3–4 ms với model dummy.

7.3 Kịch bản demo ở chế độ mô phỏng

#	Kịch bản	Kết quả mong đợi	Kết quả sim
1	Hàng tốt, QR kho A	Khay A, cnt_a tăng	PASS — SORT_A, cnt_a=1, latency 101 ms
2	Hàng tốt, QR kho B	Khay B, cnt_b tăng	PASS — SORT_B, cnt_b=1, latency 79 ms
3	Hàng rách, QR hợp lệ	Khay Loại, ảnh lỗi + nhãn	PASS — REJECT_DEFECT, nhãn tear, defect_image 13 KB
4	QR giả (chữ ký sai)	Khay Loại, alarm BAD_SIGNATURE	PASS — REJECT_SECURITY, sự kiện QR_BAD_SIGNATURE [MAJOR]
5	DỪNG KHẨN CẤP từ dashboard (RPC emergencyStop)	Băng dừng, conveyor_state = ESTOP, rpc_log	PASS — ESTOP qua RPC, rpc_audit + rpc_log; resetEstop → IDLE → RUNNING
6a	Secure OTA bản hợp lệ	Xác minh OK (phần ESP32 theo OTA_TEST.md)	PASS — manifest v3 ECDSA hợp lệ đúng thứ tự O-10…O-12
6b	OTA sửa 1 byte / giả manifest	HASH_MISMATCH / SIGNATURE_INVALID	PASS — cả hai bị từ chối đúng lý do; khôi phục OK
7	Quét lại QR đã dùng	Khay Loại, DUPLICATE	PASS — REJECT_SECURITY/DUPLICATE, sự kiện QR_DUPLICATE

#	Kịch bản	Kết quả mong đợi	Kết quả sim
8	E-stop vật lý	Dừng ngay; reset khi còn bấm bị từ chối	PASS — ESTOP(PHYSICAL); reset khi còn bấm → BAD_STATE; nhả + resetEstop → IDLE
9	Đăng nhập Viewer rồi Admin (2FA)	Viewer chỉ xem; Admin nhập 2FA	MANUAL — kiểm tra tay trên ThingsBoard
10	Mô phỏng tấn công	anomaly_flag / alarm	PASS — RPC lạ ×3 + 8 QR giả → RULE_QR_FORGERY + RULE_RPC_UNKNOWN (CRITICAL); IF cần model đã train
11	Rút dây UART khi băng chạy	FAULT LINK_LOST sau 2 s	PASS — FAULT sau 2,0 s, Pi ghi ESP_LINK_LOST; reset → IDLE → RUNNING
12	OTA ký hợp lệ nhưng phiên bản cũ	ROLLBACK_REJECTED	PASS — manifest v2 hợp lệ nhưng ≤ v3 → ROLLBACK_REJECTED

Bảng 7.3 — pi/tools/demo_check.py --mode sim: 12 PASS, 0 FAIL, 1 MANUAL trong 6,5 s

7.4 Benchmark và số liệu

Với model dummy trên laptop (ONNX Runtime 1.31, 4 luồng, ảnh 1280×960, 100 lần): mean 7,8 ms, p50 8,1 ms, p95 9,3 ms, max 11,9 ms (gồm cắt ROI, resize, chuẩn hóa; riêng session.run 0,28 ms), RAM 37 → 72 MB khi nạp model. Số này chỉ kiểm tra đường đi; model MobileNetV4 thật phải đo trên Pi 4 bằng `benchmark_model.py` và điền vào `docs/metrics.md`.

7.5 Đối chiếu tiêu chí nghiệm thu

Tiêu chí	Mục tiêu	Trạng thái
Độ chính xác AI	Acc ≥ 90 %, recall lỗi ≥ 95 %	Chờ dataset thật; evaluate.py đã chọn ngưỡng theo recall
Suy luận trên Pi	≤ 150 ms	Chờ đo trên Pi; MobileNetV4-Conv-Medium 256×256 dự kiến đạt
Dừng băng → SORT	≤ 1 s	Sim 80–130 ms (có QR); cần đo thật
Đúng khay 20/20	Kịch bản 1–4	Sim PASS; cần chạy thật
E-stop dashboard ≤ 0,5 s	—	Đường đi RPC → UART ESTOP đã test; cần quay video
QR giả/trùng bị chặn	100 %	Test đơn vị + sim PASS
OTA tấn công bị từ chối	100 % 3 kiểu	App OTA: verify_release_dir từ chối đúng lý do (31 test); ESP32 chưa chạy
Báo nhầm IF ≤ 1 / 30 phút	—	Chờ dữ liệu bình thường thật

Bảng 7.4 — Trạng thái so với tiêu chí 5.3

8. Hướng dẫn cài đặt, vận hành và demo

8.1 Chạy nhanh trên laptop (không cần phần cứng)

```
cd smart-sorter/pi
pip install -r requirements.txt           # Windows: set PYTHONUTF8=1
python -m pytest                          # toàn bộ test
python -m sorter.main --sim --config config.sim.yaml
python tools/demo_check.py                # kịch bản demo ở chế độ sim
python tools/esp_bench.py --fake          # 16 ca kiểm tra giao thức với ESP32 giả lập
cd ../ota-app && python -m pytest && python -m ota_app.main
```

8.2 Cài đặt Raspberry Pi

13. Raspberry Pi OS 64-bit Bookworm; `sudo raspi-config` bật Interface → Serial Port (login shell: No, hardware: Yes) và Camera; thêm `dtoverlay=disable-bt` vào `/boot/firmware/config.txt`.
14. `sudo bash pi/scripts/install_pi.sh`: tạo user `sorter` (video, dialout, gpio), thư mục `/etc/sorter/{secrets,certs}` 700, `/var/lib/sorter`, `/var/log/sorter`, `/opt/sorter`, venv `--system-site-packages` để dùng `python3-picamera2` từ apt, cài requirements, cài `sorter.service`.
15. `python tools/make_hmac_key.py /etc/sorter/secrets/qr_hmac.key` (quyền 600). Copy `ca.pem`, `device.pem`, `device.key` từ `thingsboard/pki/out/` vào `/etc/sorter/certs/`.
16. Copy `config.example.yaml` thành `/etc/sorter/config.yaml`, chỉnh host ThingsBoard, ROI, tốc độ băng. Copy model `packaging_cls.onnx`, `model_card.json`, `labels.json` vào `/opt/sorter/models/`.
17. `sudo systemctl enable --now sorter`; xem log `journalctl -u sorter -f` hoặc `/var/log/sorter/sorter.jsonl`.
18. Hardening: `sudo bash pi/scripts/harden_pi.sh --admin-user <user> --lan-subnet 192.168.1.0/24` rồi `bash pi/scripts/security_audit.sh`.

8.3 Nạp ESP32

```
cd smart-sorter/esp32
idf.py set-target esp32
idf.py menuconfig        # Smart Sorter: chọn cảm biến TCRT5000/HC-SR04, SORTER_HW_TEST khi kiểm tra đấu dây
idf.py build && idf.py -p /dev/ttyUSB0 flash monitor
bash tools/provision_nvs.sh nvs/wifi_ota.csv /dev/ttyUSB0   # SSID, mật khẩu, URL OTA vào NVS
python ../pi/tools/esp_bench.py --port /dev/serial0          # 16 ca kiểm tra từ Pi
```

Secure Boot V2 và Flash Encryption chỉ làm trên board dự phòng trước, theo `esp32/SECURITY.md` (ghi eFuse không đảo ngược được).

8.4 ThingsBoard

```
cd smart-sorter/thingsboard
bash pki/make_ca.sh && bash pki/make_server_cert.sh tb.local 192.168.1.10
bash pki/make_device_cert.sh sorter-pi-01 && bash pki/make_ota_server_cert.sh 192.168.1.10
cp pki/out/server.pem pki/out/server_key.pem certs/ && cp .env.example .env
docker compose up -d      # web http://localhost:8080, MQTT TLS 8883
```

Sau đó cấu hình theo `thingsboard/SETUP.md`: device profile, device X.509, rule chain, dashboard Điều khiển/Giám sát, Admin với 2FA, Viewer. Kiểm tra bằng `openssl`

`s_client -connect tb.local:8883 -CAfile ca.pem -cert device.pem -key device.key` và `nmap -p 1883` phải đóng.

8.5 Kịch bản demo bắt buộc

#	Kịch bản	Kết quả mong đợi	Chức năng
1	Hàng tốt, QR kho A	Vào khay A, bíp ngắn, cnt_a tăng, log kiện mới	E-04, P-03→P-08, T-06
2	Hàng tốt, QR kho B	Vào khay B, cnt_b tăng	E-04, P-06
3	Hàng rách, QR hợp lệ	Khay Loại, LED đỏ, ảnh lỗi + nhãn trên dashboard	AI-06, P-05, T-06
4	QR giả (chữ ký sai)	Khay Loại, đỏ nháy + còi, alarm Security alert BAD_SIGNATURE	S-03, S-09, T-05
5	Bấm DỪNG KHẨN CẤP trên dashboard	Băng dừng, conveyor_state = ESTOP, rpc_log ghi lệnh	E-08, S-05, T-04
6a	Secure OTA bản hợp lệ	ESP32 tải, xác minh, khởi động lại; fw_version mới, ota_stage SUCCESS	O-01→O-14
6b	Secure OTA bản sửa 1 byte	Từ chối HASH_MISMATCH, vẫn chạy bản cũ	O-07, O-12
7	Quét lại QR đã dùng	Khay Loại, DUPLICATE	S-03
8	Bấm E-stop vật lý (kể cả rút mạng)	Dừng ngay; phải nhả nút và Reset mới chạy lại	E-07
9	Đăng nhập Viewer rồi Admin	Viewer chỉ xem; Admin nhập mã 2FA	S-02, T-07, T-08
10	Chạy mô phỏng tấn công	anomaly_flag bật, alarm Anomaly kèm đặc trưng lệch	A-03→A-05
11	Rút dây UART khi băng chạy	ESP32 dừng sau 2 s, FAULT LINK_LOST	E-09
12	OTA ký hợp lệ nhưng phiên bản cũ	Từ chối ROLLBACK_REJECTED	O-11

Bảng 8.1 — Kịch bản demo (đặc tả 5.1, 5.2)

8.6 Chuẩn bị trước buổi demo

- Bộ hộp đánh số: 2 hộp tốt (A, B), 1 hộp rách, 1 hộp móp; QR in sẵn bằng `qr_generator.py`: hợp lệ A, hợp lệ B, QR giả (`--forge`), QR đã dùng.
- Gọi RPC resetUsedCodes và resetCounters bằng `thingsboard/tools/rpc_cli.py` ngay trước khi demo.
- Sẵn 3 bản OTA từ tab Demo của app OTA: hợp lệ, sửa 1 byte, hạ phiên bản.
- Đèn chiếu sáng bật ổn định ≥ 5 phút; đăng nhập sẵn Admin (đã qua 2FA) và Viewer trên 2 cửa sổ; board ESP32 dự phòng đã nạp cùng firmware.

9. Kết luận

9.1 Những gì đã hoàn thành

Toàn bộ 10 mô-đun của đặc tả đã được hiện thực thành mã nguồn chạy được: giao thức UART sinh tự động cho cả C và Python; firmware ESP32 đầy đủ máy trạng thái, an toàn và Secure OTA; phần mềm sorter-core trên Raspberry Pi dùng camera CSI qua Picamera2 với pipeline QR → HMAC → AI → quyết định → UART → ThingsBoard; bộ phân loại ONNX khớp notebook train của nhóm; kết nối ThingsBoard bằng mutual TLS với RPC whitelist, widget và hướng dẫn dashboard; script hardening và kiểm toán; ứng dụng Secure OTA với ba kịch bản tấn công; mô-đun Isolation Forest; công cụ kiểm thử demo và số liệu. Khoảng 25 000 dòng mã và tài liệu; 234 test tự động pass (203 phần Pi + 31 app OTA) trên máy phát triển.

9.2 Việc còn lại trước ngày chấm

19. Biên dịch và nạp firmware ESP32 bằng ESP-IDF, chạy chế độ HW_TEST rồi 16 ca `esp_bench.py` trên board thật; đối chiếu các API được đánh dấu.
20. Chụp ≥ 300 ảnh mỗi lớp tại trạm chụp thật, train bằng notebook trên Colab, đánh giá đạt recall lỗi ≥ 95 %, đo benchmark trên Pi 4.
21. Dựng ThingsBoard bằng Docker, tạo chứng chỉ, cấu hình dashboard/alarm/2FA theo SETUP.md, kết nối Pi thật qua 8883.
22. Thử Secure Boot V2 + Flash Encryption trên board dự phòng; chạy đủ 4 ca OTA và ca rollback.
23. Thu 30 phút dữ liệu bình thường, train Isolation Forest, chạy attack_sim và đánh giá.
24. Diễn tập 12 kịch bản demo ít nhất 2 lần trên mô hình hoàn chỉnh, điền số đo thật vào `docs/metrics.md`.

9.3 Hướng phát triển

- Thêm đếm kiện bằng cảm biến thứ hai ở cuối băng để xác nhận hàng đã vào đúng khay.
- Chuyển dashboard sang HTTPS và dùng ThingsBoard Edge khi triển khai nhiều trạm.
- Học chủ động (active learning): dùng ảnh lỗi lưu trên Pi để cải thiện model theo chu kỳ.
- Flash Encryption chế độ Release và ký OTA bằng HSM/khóa phần cứng cho môi trường sản xuất.

Phụ lục A — Cấu trúc mã nguồn

```
smart-sorter/
  CLAUDE.md  Makefile  README.md  .gitignore
  shared/protocol/   protocol.yaml  gen.py  gen_vectors.py  test_vectors.json
  pi/
    sorter/          config.py interfaces.py protocol_gen.py uart_codec.py uart_link.py
fake_esp32.py camera.py
```

```
                    qr_reader.py qr_security.py classifier.py classifier_onnx.py
decision.py storage.py counters.py
                    health.py security_events.py defect_images.py logging_setup.py
factories.py orchestrator.py
                    tb_client.py rpc_handler.py main.py
anomaly/{features,train,detector,rules,evaluate,store}.py
    tools/          make_hmac_key.py qr_generator.py capture_dataset.py uart_monitor.py
esp_bench.py
                    benchmark_model.py attack_sim.py demo_check.py report_metrics.py
    tests/          14 file test + fixtures/{make_images.py, images/, qr_hmac.key}
    systemd/sorter.service  scripts/{install_pi.sh, harden_pi.sh, security_audit.sh}
    config.example.yaml  config.sim.yaml  requirements.txt
  esp32/
    main/{app_main.c, version.h, Kconfig.projbuild}  partitions.csv  sdkconfig.defaults
    components/{protocol, board, motor, servo, sensor, indicator, estop, link, params, fsm,
ota_secure}
    nvs/  tools/provision_nvs.sh  README.md  OTA_TEST.md  SECURITY.md
  ai/   train_packaging_damage.ipynb  train_yolo_cls.{py,ipynb}  train_mobilenetv2.py
prepare_dataset.py
        evaluate.py  export_dummy_model.py  samples/  README.md
  thingsboard/  docker-compose.yml  pki/*.sh  widgets/  tools/{rpc_cli.py, tb_provision.py}
SETUP.md
  ota-app/  ota_app/{auth,keys,firmware,manifest,server,history,demo_attacks,main}.py  ui/
tools/verify_cli.py  tests/
  docs/  SPEC.md PROGRESS.md RUNBOOK.md traceability.md metrics.md demo_checklist.md
security_slide.md
         threat_model.md anomaly.md report/
```

Khu vực	Số dòng (xấp xỉ)
shared/protocol	730
pi/sorter (gồm anomaly)	6 650
pi/tools	2 450
pi/tests	2 510
pi/scripts	1 230
esp32	4 760
ai	1 310
thingsboard	1 400
ota-app	3 890
docs	1 720

Bảng A.1 — Thống kê dòng mã và tài liệu

Phụ lục B — Truy vết chức năng

Bảng đầy đủ 75 mã chức năng → file → test/kịch bản → trạng thái nằm trong `docs/traceability.md`. Tóm tắt theo nhóm:

Nhóm	Số mã	Hiện thực chính	Kiểm ch
E (ESP32)	14	esp32/components/{fsm,motor,servo,sensor,indicator,estop,link,params,ota_secure}	esp_benc ca; kiểm stop; bui ESP-IDF

Nhóm	Số mã	Hiện thực chính	Kiểm ch
P (Pi)	16	pi/sorter/*, pi/tools/*, systemd, install_pi.sh	pytest 20 esp_benc
AI	7	ai/*, classifier_onnx.py, benchmark_model.py, capture_dataset.py	test_class 13; evaluate/ với dumr
T (ThingsBoard)	8	docker-compose, tb_client.py, rpc_handler.py, widgets, SETUP.md	test_tb_c test_rpc_ 57; cấu h công thec SETUP.r
S (Bảo mật)	9	pki, tb_client, qr_security, rpc_handler, uart_codec, harden_pi.sh, SECURITY.md, security_events	pytest; security_ eFuse the
O (OTA)	15	ota-app/* (O-01..O-07), esp32/components/ota_secure (O-08..O-15)	ota-app 3 OTA_TE trên boar
A (Bất thường)	6	pi/sorter/anomaly/*, attack_sim.py	test_anor evaluate attack_lo

Bảng B.1 — Tóm tắt truy vết

Phụ lục C — Tài liệu tham khảo

- Đặc tả nhóm: "Smart Logistics Sorter — Đặc tả & Prompt code" (docs/SPEC.md).
- Raspberry Pi Ltd, The Picamera2 Library manual; Espressif, ESP-IDF Programming Guide v5.x (LEDC, UART, esp_ota_ops, esp_http_client, Secure Boot V2, Flash Encryption).
- ThingsBoard CE documentation: Device MQTT API, X.509 certificate authentication, RPC, Alarm rules, Two-factor authentication.
- Liu et al., Isolation Forest (ICDM 2008); Woo et al., ConvNeXt V2 (2023); Qin et al., MobileNetV4 (2024); Hinton et al., Distilling the Knowledge in a Neural Network (2015).
- RFC 2104 HMAC; RFC 6238 TOTP; FIPS 186-4 ECDSA; ONNX Runtime documentation.